

PROYECTO INTERMODULAR

PlateUp

La mejor red social de cocina multiplataforma

Módulo:	Proyecto Intermodular / 2º DAM
Autor:	Xabier López de Guereño Armada
Tutor:	Víctor Pla Soler
Centro:	IES La Vereda / La Pobla de Vallbona, València
Fecha:	25 de mayo de 2026
Repositorio:	https://github.com/XabierLDGA/PlateUp.git

I.E.S. La Vereda

La Pobla de Vallbona - València



Abstract

Castellano

PlateUp es una aplicación multiplataforma que combina el concepto de red social con el mundo de la cocina doméstica. Inspirada en plataformas como Hevy, por su sistema de registro y seguimiento de actividad, e Instagram, por su estructura social y su interfaz visual centrada en imágenes, PlateUp permite a los usuarios crear, compartir y descubrir recetas de forma dinámica, interactiva y divertida. La aplicación dispone de un feed social con filtros por categoría, un sistema de likes y follows entre usuarios, un módulo de gamificación con logros y retos, un modo guiado de cocina paso a paso con temporizadores, y una versión Android nativa generada mediante Capacitor. El backend está implementado con Java y Spring Boot 3, protegido con Spring Security y autenticación JWT, y se conecta a una base de datos MySQL alojada en Railway. El conjunto se despliega en producción mediante Railway.

English

PlateUp is a cross-platform application that combines the concept of a social network with the world of home cooking. Inspired by platforms such as Hevy, for its activity tracking system, and Instagram, for its social structure and image-centric visual interface, PlateUp allows users to create, share, and discover recipes in a dynamic, interactive and enjoyable way. The application includes a social feed with category filters, a system of likes and follows between users, a gamification module with achievements and challenges, a step-by-step guided cooking mode with timers, and a native Android version generated through Capacitor. The backend is implemented with Java and Spring Boot 3, protected with Spring Security and JWT authentication, and connects to a MySQL database hosted on Railway. The whole system is deployed in production via Railway.

Valencià

PlateUp és una aplicació multiplataforma que combina el concepte de xarxa social amb el món de la cuina domèstica. Inspirada en plataformes com Hevy, pel seu sistema de registre i seguiment d'activitat, i Instagram, per la seua estructura social i interfície visual centrada en imatges, PlateUp permet als usuaris crear, compartir i descobrir receptes de manera dinàmica, interactiva i divertida. L'aplicació disposa d'un feed social amb filtres per categoria, un sistema de likes i follows entre usuaris, un mòdul de gamificació amb assoliments i reptes, un mode guiat de cuina pas a pas amb temporitzadors, i una versió Android nativa generada amb Capacitor. El backend està implementat amb Java i Spring Boot 3, protegit amb Spring Security i autenticació JWT, i es connecta a una base de dades MySQL allotjada a Railway. El conjunt es desplega en producció mitjançant Railway.

Índice

1. Introducción.....	7
1.1 Breve descripción del proyecto.....	7
1.2 Justificación del proyecto.....	7
1.3 Objetivos del proyecto.....	7
1.4 Alcance del proyecto.....	8
1.5 Planificación temporal.....	9
2. Módulos a los que implica.....	10
2.1 Módulos de 1º DAM.....	10
2.2 Módulos de 2º DAM.....	11
3. Contexto empresarial y estudio de mercado.....	14
3.1 Creación de la empresa ficticia.....	14
3.2 Sector de actividad.....	14
3.3 Misión, visión y valores.....	14
Misión.....	14
Visión.....	14
Valores.....	14
3.4 Público objetivo.....	15
3.5 Estudio de mercado y análisis de la competencia.....	15
3.6 Análisis DAFO.....	16
3.7 Viabilidad del proyecto.....	16
Viabilidad técnica.....	16
Viabilidad económica.....	17
Viabilidad comercial.....	17
3.8 Modelo de negocio.....	17
3.9 Prevención de riesgos laborales, salud y SGE.....	17
Prevención de riesgos laborales.....	17
Salud mental.....	17
Sistemas de Gestión Empresarial.....	17
4. Análisis técnico.....	18
4.1 Arquitectura del sistema.....	18
4.2 Stack tecnológico y justificación de las elecciones.....	18
Java 21 + Spring Boot 3.5.6 (Backend).....	18
Spring Security + JWT (Autenticación).....	19
Spring Data JPA + Hibernate (Persistencia).....	19
MySQL 8 en Railway (Base de datos).....	19
Vue.js 3 + Composition API (Frontend).....	19
Tailwind CSS v4 (Estilos).....	19
Vite (Bundler).....	19
Axios (Cliente HTTP).....	19
Pinia (Gestión del estado).....	19
Capacitor (Android).....	20
Docker + Railway (Despliegue).....	20
4.3 Control de versiones.....	20
4.4 Seguridad de la aplicación.....	20
Autenticación con JWT.....	20
Filtro JWT en Spring Security.....	20
Política de acceso a endpoints.....	20
Cifrado de contraseñas.....	21

4.5 Herramientas utilizadas durante el desarrollo.....	21
5. Análisis del sistema.....	22
5.1 Actores del sistema.....	22
Usuario no autenticado.....	22
Usuario autenticado.....	22
Administrador.....	22
5.3 Requisitos funcionales.....	22
5.4 Requisitos no funcionales.....	23
Rendimiento.....	23
Usabilidad.....	23
Seguridad.....	23
Disponibilidad.....	23
Mantenibilidad.....	24
Escalabilidad.....	24
Compatibilidad.....	24
5.5 Historias de usuario.....	24
Autenticación y gestión de cuenta.....	24
Gestión de recetas.....	25
Interacción social.....	25
Modo cocina y gamificación.....	25
Administración.....	26
6. Desarrollo técnico detallado.....	27
6.1 Metodología de trabajo.....	27
6.2 Desarrollo del backend (Spring Boot + API REST).....	27
Estructura del proyecto.....	27
Controladores REST implementados.....	27
Documentación automática con Springdoc OpenAPI.....	28
6.3 Desarrollo del frontend (Vue.js 3).....	28
Estructura del proyecto frontend.....	28
Interceptor de Axios.....	28
Guard de navegación en Vue Router.....	28
6.4 Aplicación móvil Android con Capacitor.....	28
6.5 Sistema de subida de imágenes.....	29
6.6 Sistema de gamificación.....	29
6.7 Pruebas realizadas.....	29
Pruebas de la aplicación web.....	30
Pruebas de la aplicación Android.....	30
Pruebas con usuarios reales.....	30
6.8 Despliegue en producción (Railway).....	30
Base de datos MySQL gestionada.....	30
Backend (API REST).....	30
Frontend (Sitio estático).....	30
Docker Compose para desarrollo local.....	30
7. Base de datos.....	31
7.1 Diseño conceptual.....	31
7.2 Descripción de las entidades.....	31
7.3 Paso a tablas (modelo relacional completo).....	31
8. Diseño visual del proyecto (UX/UI).....	34
8.1 Filosofía de diseño y decisiones generales.....	34
8.2 AuthView — Pantalla de autenticación.....	34
8.3 HomeView — Feed principal.....	34
8.4 ExploreView — Exploración y búsqueda.....	34

8.5 RecipeDetailView — Detalle de receta.....	35
8.6 CookingView — Modo cocina guiado.....	35
8.7 CreateView — Creación de receta.....	35
8.8 ProfileView — Perfil propio.....	35
8.9 UserProfileView — Perfil de otros usuarios.....	35
8.10 EditProfileView — Edición del perfil.....	35
8.11 ConnectionsView — Seguidores y seguidos.....	36
8.12 AdminDashboardView — Panel de administración.....	36
9. Mejoras a largo plazo.....	37
9.1 Sistema de notificaciones en tiempo real.....	37
9.2 Recomendaciones personalizadas con IA.....	37
9.3 Integración con compra de ingredientes.....	37
9.4 Versión iOS con Capacitor.....	37
9.5 Mensajería directa entre usuarios.....	37
9.6 Mejoras en el sistema de gamificación.....	37
9.7 Internacionalización (i18n).....	38
9.8 Almacenamiento de imágenes en CDN.....	38
9.9 Sistema de reportes y moderación.....	38
9.10 Perfiles públicos y privados con sistema de solicitudes de seguimiento.....	38
9.11 Implementación de refresh tokens.....	38
10. Conclusión.....	39
11. Bibliografía y fuentes consultadas.....	40
12. Anexos.....	41
Anexo I. Estructura completa del repositorio.....	41
Anexo II. Endpoints completos de la API REST.....	42
Anexo III. Variables de entorno.....	44
Anexo IV. Dependencias del backend (pom.xml).....	45
Anexo V. Dependencias del frontend (package.json).....	46
Anexo VI. Modelo entidad-relación.....	47
Anexo VII. Identidad visual: paleta de colores y tipografía.....	48
Filosofía visual.....	48
Paleta de colores.....	48
Tipografía.....	49
Tokens de forma — border-radius.....	49
Elevación y separación.....	49
Anexo VIII. Capturas de pantalla de la interfaz.....	50
AuthView (Pantalla de autenticación).....	50
HomeView (Feed principal).....	51
ExploreView (Exploración y búsqueda).....	52
RecipeDetailView (Detalle de receta).....	53
CookingView (Modo cocina guiado).....	54
CreateView (Creación de receta).....	55
ProfileView (Perfil propio).....	56
ConnectionsView (Panel de followers).....	57
AdminDashboardView (Panel de administración).....	58
Anexo IX. Repositorio de GitHub.....	59
Estructura del repositorio.....	59
Historial de commits y evolución del proyecto.....	59
Anexo X. Pruebas con Swagger UI.....	60
Organización de la colección.....	60
Casos de prueba principales.....	60
Resultados de las pruebas.....	61

1. Introducción

1.1 Breve descripción del proyecto

PlateUp es una aplicación móvil y plataforma web orientada a la gastronomía que fusiona el concepto de red social con una plataforma completa de gestión y descubrimiento de recetas. El proyecto surge de la necesidad de crear un espacio donde los usuarios puedan no solo consultar recetas, sino también publicarlas, interactuar con otros, seguir a sus cocineros favoritos y formar parte de una comunidad activa alrededor de la cocina.

La aplicación se ha desarrollado con un stack moderno: backend en Java 21 con Spring Boot 3.5.6 y MySQL; frontend en Vue.js 3 con Tailwind CSS v4 y Vite; y versión Android generada con Capacitor. El sistema completo se despliega en Railway, que aloja backend, frontend y base de datos.

PlateUp no es únicamente una aplicación de recetas: es una red social especializada donde el contenido gastronómico es el protagonista. Los usuarios tienen perfil propio, pueden seguir a otras personas y recibir seguidores, explorar el contenido de la comunidad, participar en desafíos culinarios y desbloquear logros que premian la participación activa.

1.2 Justificación del proyecto

La motivación principal surge de un hueco claro en el mercado: existen aplicaciones de recetas tradicionales (contenido estático, sin interacción social) y redes sociales generalistas como Instagram o TikTok (donde el contenido de cocina convive desordenado con todo tipo de publicaciones), pero falta una solución intermedia.

PlateUp propone una plataforma donde la receta tiene una estructura formal completa (título, descripción, imagen, ingredientes con cantidades, pasos numerados, dificultad, tiempo y categoría) e incorpora a la vez todos los elementos sociales que hacen volver al usuario: seguimiento, feed personalizado, likes, comentarios y descubrimiento.

Desde el punto de vista técnico, el proyecto está plenamente justificado como ejercicio académico integrador, ya que exige aplicar conocimientos de POO, diseño de bases de datos relacionales, desarrollo de APIs REST, seguridad con JWT, desarrollo frontend reactivo y distribución multiplataforma, abarcando prácticamente todos los módulos del ciclo DAM.

1.3 Objetivos del proyecto

Objetivo general

Desarrollar una aplicación completa llamada PlateUp, orientada al ámbito de la gastronomía, que combine el concepto de red social con una plataforma estructurada de recetas de cocina, operativa tanto en entorno web como en dispositivo móvil Android, con arquitectura profesional, segura y preparada para un entorno real de producción.

Objetivos específicos

- Diseñar e implementar un sistema completo de gestión de usuarios con registro, inicio de sesión, gestión de perfil, edición de datos personales y subida de imagen de avatar.

- Desarrollar el módulo de recetas completo con todos los campos relevantes: título, descripción, imagen, tiempo de preparación, número de raciones, nivel de dificultad, categoría, lista de ingredientes con cantidades y unidades, y pasos de preparación ordenados.
- Implementar un sistema de interacción social con likes, follows, feed dinámico de recetas ordenado por fecha y sección de exploración para descubrir nuevo contenido.
- Incorporar un sistema de categorías de recetas con soporte para las categorías: Vegan, Vegetarian, Meat, Fish, Pasta, Rice, Soup, Salad, Breakfast, Dessert, Quick, Student, Healthy y Snack.
- Añadir un sistema de gamificación con logros y retos que incentive la participación activa de los usuarios.
- Desarrollar un modo cocina guiado paso a paso con temporizadores integrados por cada paso de la receta.
- Garantizar la seguridad del sistema mediante Spring Security con política stateless y autenticación mediante JSON Web Tokens.
- Conseguir que la aplicación funcione en entorno web moderno y en dispositivo Android real mediante Capacitor.
- Desplegar el sistema en producción con Railway, configurando correctamente las variables de entorno y los servicios Docker.
- Generar documentación automática de la API REST mediante Springdoc OpenAPI.

1.4 Alcance del proyecto

El proyecto PlateUp incluye el desarrollo completo de los siguientes componentes:

- Aplicación web SPA (Single Page Application) desarrollada con Vue.js 3, Vite, Tailwind CSS v4 y Axios.
- Backend REST API desarrollado con Java 21, Spring Boot 3.5.6, Spring Security y Spring Data JPA.
- Base de datos relacional MySQL 8 con un modelo de datos normalizado con más de quince tablas relacionadas.
- Sistema de autenticación stateless con JWT (JSON Web Tokens) con validez de 30 días.
- Módulo de subida de imágenes (recetas y avatares) con almacenamiento en servidor.
- Panel de administración con estadísticas y gráficos de uso de la plataforma.
- Aplicación Android generada mediante Capacitor, instalable en dispositivos reales.
- Despliegue en producción mediante Docker y Railway.

El proyecto no incluye en su versión actual: sistema de mensajería directa entre usuarios, integración con pasarela de pagos, recomendaciones basadas en inteligencia artificial, notificaciones push en tiempo real ni sistema de reportes de contenido inapropiado (aunque la tabla Reports está definida en la base de datos para uso futuro).

1.5 Planificación temporal

El desarrollo de PlateUp se organizó en fases incrementales durante aproximadamente doce semanas, siguiendo una metodología de desarrollo iterativa donde cada fase aportaba una versión funcional sobre la que continuar construyendo.

Fase	Actividades principales
Semanas 1 - 3 (Septiembre)	Análisis de requisitos, diseño del modelo entidad-relación, definición de la arquitectura del sistema y configuración del entorno de desarrollo.
Semanas 4 - 10 (Octubre - Noviembre)	Creación del proyecto Spring Boot, implementación del modelo de datos con JPA, desarrollo de repositorios y controladores REST principales.
Semanas 11 - 22 (Diciembre - Febrero)	Ampliación del backend con todos los controladores, pruebas de la API con Swagger UI y configuración de Docker y Docker Compose.
Semanas 23 - 27 (Marzo)	Desarrollo del frontend en Vue.js: estructura de vistas, conexión con la API mediante Axios, gestión del estado con Pinia y navegación con Vue Router.
Semanas 28 - 30 (Abril)	Mejora visual completa con Tailwind CSS v4, diseño responsive y pulido de la interfaz de todas las pantallas.
Semanas 31 - 33 (Abril - Mayo)	Implementación de Spring Security con JWT, cifrado BCrypt, protección de endpoints y pruebas de seguridad.
Semanas 34-36 (Mayo)	Últimos retoques funcionales, versión Android con Capacitor, despliegue en Railway y elaboración de la memoria.

2. Módulos a los que implica

El proyecto PlateUp integra de forma transversal los conocimientos adquiridos en los módulos cursados durante el ciclo formativo de Grado Superior en Desarrollo de Aplicaciones Multiplataforma. A continuación se detalla la relación de cada módulo con el proyecto, tanto de primero como de segundo curso.

2.1 Módulos de 1º DAM

Módulo	Aplicación en PlateUp
Programación	PlateUp aplica los principios de POO en Java 21 a lo largo de todas las capas del backend: las entidades (User, Recipe, Ingredient, Follow, Like, Achievement, etc.) son clases encapsuladas con atributos privados y accesores; la lógica de negocio se organiza en clases Service que implementan interfaces, desacoplando la presentación de la persistencia; se emplean colecciones genéricas (List, Optional, Set), excepciones personalizadas (ResourceNotFoundException, AuthenticationRequiredException) y características de Java moderno como anotaciones, genéricos e inyección de dependencias mediante Spring Boot.
Bases de Datos	PlateUp parte de un modelo relacional normalizado en tercera forma normal con más de quince tablas interrelacionadas. El diseño DDL utiliza tipos apropiados (BIGINT para identificadores, VARCHAR con longitud máxima, DATETIME, ENUM), claves primarias simples y compuestas, claves foráneas con ON DELETE CASCADE para mantener la integridad referencial, índices en campos frecuentemente consultados y restricciones UNIQUE en username y email. Los scripts SQL incluyen SELECT con múltiples JOIN, INSERT masivo de datos de prueba y operaciones UPDATE/DELETE controladas.
Lenguajes de Marcas	JSON es el formato de intercambio entre el frontend Vue.js y la API REST del backend. La aplicación web utiliza HTML semántico (header, nav, main, section, article); el AndroidManifest.xml define permisos e intenciones; los archivos YAML (docker-compose.yml, railway.toml) y .properties permiten parametrizar el sistema sin tocar código, y los .env gestionan variables sensibles. La validación con Spring Validation asegura

	que los datos entrantes cumplen las restricciones declaradas (campos requeridos, formato de email, longitudes máximas).
Sistemas Informáticos	Se trabajó con el sistema de archivos para la subida y almacenamiento de imágenes mediante FileStorageService, y se configuraron entornos de desarrollo en Windows con JDK 21, Node.js, MySQL Server y Android Studio. Se gestionaron los puertos sin conflictos (8080 backend, 5173 Vite, 3306 MySQL), reglas de firewall local y variables de entorno del sistema para credenciales y claves de API, manteniéndolas fuera del repositorio.
Entornos de Desarrollo	IntelliJ IDEA fue el IDE principal del backend Java, con sus herramientas de refactorización, depuración y gestión Maven. Visual Studio Code se utilizó para el frontend Vue.js con extensiones específicas. Las pruebas se realizaron sistemáticamente con Swagger UI verificando códigos HTTP y datos de respuesta, y se elaboraron diagramas de clases y de flujo (autenticación JWT, creación de receta) para documentar la arquitectura.
FOL	El proyecto incorpora la prevención de riesgos laborales identificando los propios del trabajo prolongado con ordenadores (fatiga visual, problemas posturales, túnel carpiano, estrés) y aplicando medidas como pausas activas, configuración ergonómica del puesto y ajuste de pantalla. En cuanto a la protección de datos personales, las contraseñas se cifran con BCrypt, se limitan los datos a los estrictamente necesarios y el usuario mantiene control sobre su perfil y contenido, alineándose con GDPR y LOPD.

2.2 Módulos de 2º DAM

Módulo	Aplicación en PlateUp
Programación Multimedia y Dispositivos Móviles	Capacitor se eligió frente a React Native o Flutter por permitir reutilizar el código Vue.js sin reescribirlo. El archivo capacitor.config.json define el identificador de paquete (com.plateup.app) y la configuración Android. El AndroidManifest.xml declara los permisos necesarios

	(INTERNET, READ/WRITE_EXTERNAL_STORAGE) y la sincronización se realiza con npx cap sync android. La compilación del APK desde Android Studio se probó en dispositivo físico, verificando navegación, subida de imágenes desde galería y comunicación con la API.
Acceso a Datos	La capa de persistencia se implementa con Spring Data JPA e Hibernate ORM. Las entidades se anotan con @Entity y sus relaciones con @OneToMany, @ManyToOne y @ManyToMany; las claves primarias compuestas se modelan con @EmbeddedId (Follows, Likes). Los repositorios extienden JpaRepository<Entidad, Clave> y declaran métodos por convención de nombres (findByUsername, existsByFollowerIdAndFollowedId), evitando código JDBC manual. HikariCP gestiona el pool de conexiones contra MySQL en Railway, y FileStorageService encapsula el acceso al sistema de archivos para imágenes.
Programación de Servicios y Procesos	El backend de PlateUp expone una API REST con 18 controladores y comunicación HTTP estándar (GET, POST, PUT, DELETE). El cliente Vue.js se conecta mediante Axios con un interceptor que añade automáticamente el token JWT a cada petición, y las respuestas JSON se deserializan a objetos JavaScript. La seguridad cubre autenticación JWT con HMAC-SHA256, contraseñas cifradas con BCrypt (salt aleatorio por hash), control de acceso por endpoint con Spring Security, protección CORS configurada, validación con @Valid en los DTOs y comunicación HTTPS en producción. El rendimiento se optimiza con respuestas compactas, paginación e índices en base de datos.
Desarrollo de Interfaces	La interfaz se desarrolla con Vue.js 3 utilizando Single File Components (template, script y estilos integrados) y Tailwind CSS v4 para estilos utility-first. Se implementaron once vistas (HomeView, ExploreView, RecipeDetailView, CreateView, ProfileView, EditProfileView, UserProfileView, ConnectionsView, CookingView, AuthView, AdminDashboardView) y componentes reutilizables como RecipeCard y BottomNav. El diseño es mobile-first con áreas de toque generosas (mínimo 44x44px), feedback inmediato y navegación persistente. Se cumplen criterios de accesibilidad WCAG en contraste y legibilidad. La versión web se sirve como sitio estático con HTTPS, y la

	versión Android empaqueta la misma aplicación en un APK mediante Capacitor. Las pruebas se realizaron en navegadores modernos y dispositivo Android físico.
Sistemas de Gestión Empresarial (SGE)	PlateUp implementa un sistema de gestión centralizado sobre MySQL, comparable a un ERP en miniatura: la base de datos es el núcleo, y el panel de administración aporta funcionalidades de consulta y análisis similares a las de un módulo de reporting. Los KPIs (usuarios, recetas, likes, comentarios) y los gráficos de distribución por categoría, evolución mensual y ranking de usuarios facilitan la toma de decisiones basada en datos. El modelo de negocio freemium (publicidad, suscripción premium, colaboraciones) sigue los mismos principios de monetización de muchos sistemas empresariales reales.
IPE II	PlateUp se aborda como una propuesta empresarial completa. El análisis DAFO identifica fortalezas (combinación única de red social y plataforma de recetas, diseño moderno, gamificación), debilidades (proyecto nuevo sin base de usuarios), oportunidades (crecimiento del mercado food-tech, interés en cocina casera) y amenazas (competencia de Cookpad, Instagram). El estudio de mercado analiza competencia directa e indirecta (Cookpad, Tasty, Instagram, Pinterest) y define un modelo freemium con múltiples fuentes de ingreso. El análisis de viabilidad confirma que el proyecto es técnica, económica y comercialmente viable, y la metodología Design Thinking se aplica desde la identificación de la necesidad hasta la validación mediante pruebas de usuario.

3. Contexto empresarial y estudio de mercado

3.1 Creación de la empresa ficticia

Para dotar al proyecto de un enfoque más realista, se plantea la creación de una empresa ficticia denominada PlateUp Labs S.L., una startup tecnológica con sede en Valencia dedicada a la gastronomía social. La empresa se organiza con un equipo multidisciplinar pequeño y ágil (un desarrollador fullstack, un diseñador UX/UI y un responsable de producto y comunicación), con estructura organizativa plana que favorece la toma de decisiones rápida.

La empresa se organiza con un equipo multidisciplinar pequeño y ágil, típico de una startup en fase inicial: un desarrollador fullstack con perfiles tanto en backend como en frontend, un diseñador UX/UI, y un responsable de producto y comunicación. La estructura organizativa es plana, lo que favorece la toma de decisiones rápida y la adaptación constante al mercado.

3.2 Sector de actividad

PlateUp Labs opera en el sector tecnológico, en el desarrollo de aplicaciones móviles y plataformas digitales orientadas a la gastronomía y las redes sociales (food-tech / social food), un sector con crecimiento sostenido impulsado por el contenido gastronómico de YouTube, Instagram, TikTok y Pinterest.

Según datos del sector, el mercado de apps de cocina supera los 600 millones de dólares a nivel mundial con un CAGR superior al 8%. El segmento objetivo de PlateUp (18-35 años con interés en cocina y activos en redes sociales) es uno de los grupos demográficos más activos en consumo digital.

3.3 Misión, visión y valores

Misión

La misión de PlateUp Labs es facilitar una forma más social, intuitiva y motivadora de compartir recetas de cocina, conectando a personas con intereses gastronómicos comunes dentro de una misma plataforma digital. Queremos transformar la experiencia de cocinar en casa en un acto comunitario, donde cada receta sea una oportunidad de aprender, compartir e inspirarse con otros.

Visión

Convertirnos en la plataforma de referencia para cocineros aficionados en el mercado hispanohablante, expandirnos al mercado europeo y posicionarnos como la red social especializada en gastronomía con mayor tasa de retención e interacción. A largo plazo, PlateUp aspira a integrar recomendaciones con IA y a establecer alianzas con marcas de alimentación y supermercados online.

Valores

- Comunidad: la plataforma existe para las personas y las conexiones entre ellas.

- **Accesibilidad:** cocinar bien no debe ser exclusivo de expertos ni de personas con recursos económicos elevados.
- **Innovación:** buscamos constantemente nuevas formas de mejorar la experiencia del usuario.
- **Calidad:** tanto en el código como en el diseño, siempre buscamos la excelencia.
- **Transparencia:** somos honestos con nuestros usuarios sobre el uso de sus datos y la publicidad.

3.4 Público objetivo

El público objetivo principal de PlateUp está formado por personas jóvenes y adultas con acceso a smartphone, que utilizan habitualmente redes sociales y que muestran interés por la cocina, la alimentación y el descubrimiento de nuevas recetas. Concretamente, se identifican los siguientes perfiles de usuario:

- **Estudiantes universitarios (18-25 años):** buscan recetas rápidas, económicas y fáciles de preparar con los ingredientes que tienen en casa.
- **Personas que viven solas o en piso compartido (22-32 años):** necesitan organizar sus comidas semanales y buscan inspiración para no repetir siempre lo mismo. Valoran la estructura clara de ingredientes y pasos.
- **Aficionados a la cocina saludable (25-40 años):** interesados en las categorías Vegan, Vegetarian y Healthy. Siguen a influencers de alimentación y comparten sus propias recetas.
- **Cocineros amateur creativos (20-35 años):** disfrutan creando recetas propias y quieren compartirlas con una comunidad que las valore.
- **Familias que buscan nuevas ideas para el menú semanal (30-45 años):** utilizan la plataforma principalmente para descubrir recetas y seguir a cocineros de confianza.

3.5 Estudio de mercado y análisis de la competencia

El análisis competitivo de PlateUp identifica los siguientes tipos de competidores directos e indirectos:

Competidor	Análisis
Cookpad	La plataforma de recetas más grande del mundo. Punto fuerte: gran volumen de recetas. Debilidad: interfaz anticuada, poca interacción social real.
Tasty (BuzzFeed)	Recetas en formato vídeo muy viral. Punto fuerte: contenido visual atractivo. Debilidad: no hay perfiles de usuario reales ni sistema de follows.
Instagram / TikTok	Redes sociales generalistas con mucho contenido de cocina. Punto fuerte: alcance masivo. Debilidad: no existe estructura de receta; el contenido es efímero y desorganizado.
Pinterest	Plataforma de descubrimiento visual. Punto fuerte: inspiración visual. Debilidad: redirige a blogs externos, sin comunidad propia ni interacción directa.

PlateUp se diferencia de todos estos competidores por ofrecer simultáneamente: estructura completa de receta con ingredientes y pasos detallados, sistema social completo con follows y likes, perfiles de usuario personalizados con estadísticas, categorías bien organizadas, sistema de gamificación y modo cocina guiado, todo dentro de una misma aplicación con diseño mobile-first.

3.6 Análisis DAFO

Cuadrante	Contenido
Fortalezas	Combinación única de red social y plataforma de recetas en una sola herramienta. Diseño moderno, limpio y centrado en experiencia móvil. Sistema de gamificación que incentiva la participación. Arquitectura técnica sólida y escalable. Distribución multiplataforma (web + Android). Modo cocina guiado paso a paso con temporizadores.
Debilidades	Proyecto de nueva creación sin reconocimiento de marca ni base de usuarios establecida. Recursos económicos y humanos limitados propios de una startup inicial. Necesidad de masa crítica de usuarios para que la parte social sea atractiva. Sin versión iOS en la versión actual. Almacenamiento de imágenes en servidor local (no CDN).
Oportunidades	Crecimiento sostenido del mercado food-tech a nivel global. Aumento del interés por la cocina casera y la alimentación saludable tras la pandemia. Escasez de plataformas especializadas con buena experiencia móvil en español. Posibilidad de integración con IA para recomendaciones personalizadas. Alianzas potenciales con marcas de alimentación y supermercados online.
Amenazas	Gran presencia de plataformas ya establecidas con millones de usuarios. Las redes sociales generalistas como TikTok e Instagram copan gran parte de la atención gastronómica. Cambios rápidos en las tendencias de consumo digital. Riesgo de que competidores grandes copien las funcionalidades diferenciadas de PlateUp.

3.7 Viabilidad del proyecto

Viabilidad técnica

El proyecto es técnicamente viable: todas las tecnologías empleadas son maduras y ampliamente documentadas. Java y Spring Boot son el estándar de facto en backend empresarial; Vue.js es uno de los frameworks frontend más adoptados; MySQL es una de las bases de datos más utilizadas; y Capacitor, mantenido por Ionic, tiene soporte activo para Android e iOS. El stack es coherente, escalable y adecuado para el tipo de aplicación desarrollada.

Viabilidad económica

El coste de desarrollo inicial es muy bajo: todas las herramientas son gratuitas y de código abierto. El plan gratuito de Railway permite tener backend, frontend y MySQL accesibles online sin coste durante la fase de validación. A medida que la plataforma escale, los costes crecerán de forma proporcional al uso, modelo habitual en aplicaciones SaaS modernas.

Viabilidad comercial

El mercado objetivo está claramente identificado y tiene un volumen significativo. La propuesta de valor de PlateUp es diferenciada respecto a los competidores existentes. El modelo de negocio freemium es viable desde el primer día y permite crecer sin barreras de entrada para los usuarios. La monetización puede activarse gradualmente a medida que la base de usuarios crezca.

3.8 Modelo de negocio

PlateUp adopta un modelo de negocio freemium con las siguientes fuentes de ingresos potenciales:

- Publicidad no intrusiva: anuncios contextuales relacionados con ingredientes, utensilios de cocina y productos alimentarios para usuarios del plan gratuito.
- Suscripción Premium mensual/anual: acceso sin publicidad, colecciones privadas ilimitadas, estadísticas avanzadas del perfil, insignias exclusivas y acceso anticipado a nuevas funcionalidades.
- Recetas y colecciones de chefs reconocidos: contenido premium de cocineros profesionales disponible por suscripción.
- Colaboraciones con marcas de alimentación: recetas patrocinadas por marcas del sector, perfectamente etiquetadas como contenido patrocinado.
- Integración con supermercados online: comisión por la compra de ingredientes directamente desde la lista de una receta.

3.9 Prevención de riesgos laborales, salud y SGE

Prevención de riesgos laborales

Se identificaron riesgos típicos de programación (fatiga visual, problemas posturales, síndrome del túnel carpiano, fatiga mental). Las medidas aplicadas fueron: configuración ergonómica del puesto, pausas activas cada 45-60 minutos, técnica Pomodoro, y pantalla con brillo/color adaptados. En seguridad informática: cifrado BCrypt, autenticación JWT, validación de datos, CORS restrictivo y variables de entorno para credenciales.

Salud mental

Se gestionó el estrés, frustración y bloqueos mediante: división del proyecto en tareas pequeñas, commits frecuentes, lista de tareas priorizada, comunicación con el tutor y aceptación de limitaciones.

Sistemas de Gestión Empresarial

PlateUp funciona como un sistema centralizado con MySQL como núcleo. El panel de administración ofrece reportes y analytics similares a un ERP, mostrando métricas clave (usuarios, recetas, likes, crecimiento).

4. Análisis técnico

4.1 Arquitectura del sistema

PlateUp sigue una arquitectura de tres capas bien diferenciadas (cliente, servidor y base de datos), con separación total entre frontend y backend. La comunicación entre las capas se realiza exclusivamente mediante una API REST que devuelve y recibe datos en formato JSON. Esta arquitectura se conoce como SPA + REST API y es una de las más adoptadas en el desarrollo de aplicaciones web y móviles modernas.

Componente	Función
Cliente web (Vue.js SPA)	Aplicación de página única que se ejecuta en el navegador del usuario. Gestiona la interfaz, la navegación y el estado. Se comunica con el backend exclusivamente mediante llamadas HTTP asíncronas con Axios.
Cliente Android (Capacitor)	Wrapper nativo que empaqueta la aplicación Vue.js como una app Android. Permite acceder a APIs nativas del dispositivo como cámara y almacenamiento.
Backend REST (Spring Boot)	Servidor Java que expone la API REST. Gestiona la autenticación, la lógica de negocio y el acceso a datos. Organizado en capas: Controller → Service → Repository → Entity.
Base de datos (MySQL en Railway)	Base de datos relacional gestionada en la nube mediante Railway. Almacena toda la información persistente del sistema con alta disponibilidad.
Almacenamiento de imágenes	Sistema de subida de archivos con almacenamiento en el servidor de Railway. Las URLs de las imágenes se guardan en la base de datos para ser recuperadas por el frontend.

4.2 Stack tecnológico y justificación de las elecciones

Cada tecnología del stack de PlateUp fue elegida por razones concretas, evaluando alternativas y seleccionando la opción más adecuada para los requisitos del proyecto.

Java 21 + Spring Boot 3.5.6 (Backend)

Java fue elegido por ser el lenguaje principal del ciclo DAM y por su solidez en aplicaciones empresariales. La versión 21 aporta records y pattern matching. Spring Boot se eligió frente a Micronaut o Quarkus por ser el framework Java más utilizado, con la mejor documentación e integración con Spring Security y Spring Data JPA. La versión 3.x aporta soporte para Jakarta EE 10 y Java 17+ con mejoras de rendimiento.

Spring Security + JWT (Autenticación)

Spring Security es el estándar de seguridad para aplicaciones Spring; se eligió sobre Auth0 o Firebase por el control total sobre la autenticación sin depender de servicios externos. JWT se eligió sobre las sesiones de servidor por ser stateless: cada petición lleva toda la información necesaria para autenticar, lo que permite escalado horizontal sin sesiones compartidas. Se utilizó JJWT 0.12.5, la implementación más madura para Java.

Spring Data JPA + Hibernate (Persistencia)

Spring Data JPA se eligió por su integración con Spring Boot y por eliminar el código JDBC manual. Hibernate actúa como proveedor JPA gestionando el mapeo objeto-relacional. Permite desarrollar la capa de acceso a datos declarativamente, mediante interfaces Repository con métodos derivados del nombre (findByUsername, findByRecipeld, etc.), reduciendo el código repetitivo.

MySQL 8 en Railway (Base de datos)

MySQL fue elegido frente a PostgreSQL o H2 por ser la base de datos relacional más utilizada y por la familiarización adquirida en el módulo de Bases de Datos. La versión 8 mejora el rendimiento, soporta el estándar SQL y gestiona el charset utf8mb4 (necesario para emojis). Railway se eligió como proveedor por su plan gratuito con alta disponibilidad, copias de seguridad automáticas y conexión SSL.

Vue.js 3 + Composition API (Frontend)

Vue.js 3 fue elegido frente a React o Angular por su curva de aprendizaje más suave, sintaxis clara y excelente documentación en español. La Composition API permite organizar el código por funcionalidades, mejorando la reutilización. Los Single File Components agrupan template, script y estilos en un mismo archivo, facilitando la organización del proyecto.

Tailwind CSS v4 (Estilos)

Tailwind CSS fue elegido frente a Bootstrap o Bulma por su enfoque utility-first que permite construir interfaces complejas directamente en el HTML del componente. La versión 4 mejora el rendimiento de compilación mediante Lightning CSS. Frente a Bootstrap, ofrece más flexibilidad para diseños personalizados sin sobrescribir estilos predefinidos.

Vite (Bundler)

Vite fue elegido frente a Webpack por su velocidad de arranque y configuración más sencilla. Utiliza módulos ES nativos del navegador en desarrollo, eliminando el paso de bundling y reduciendo el tiempo de recarga. En producción genera un bundle optimizado con Rollup. Es la herramienta recomendada por el equipo de Vue.js.

Axios (Cliente HTTP)

Axios fue elegido sobre la API Fetch nativa del navegador por su mejor manejo de errores, su soporte para interceptores HTTP (que en PlateUp se utilizan para añadir automáticamente el token JWT a todas las peticiones) y su compatibilidad con el entorno de Capacitor en Android.

Pinia (Gestión del estado)

Pinia es la librería de gestión del estado oficial recomendada por el equipo de Vue.js para Vue 3, sustituyendo a Vuex. Se eligió por su API más simple e intuitiva, su integración nativa

con Vue DevTools y su soporte completo para TypeScript. En PlateUp, Pinia gestiona el estado de autenticación del usuario mediante authStore.js.

Capacitor (Android)

Capacitor fue elegido sobre React Native o Flutter porque permite reutilizar el código Vue.js sin reescribirlo, reduciendo drásticamente el tiempo de desarrollo de la versión móvil. Envuelve la aplicación web en un WebView nativo y proporciona acceso a APIs nativas mediante plugins. Es mantenido por Ionic.

Docker + Railway (Despliegue)

Docker se eligió para el despliegue porque garantiza un comportamiento idéntico en cualquier entorno. Railway se eligió como plataforma por ofrecer un plan gratuito que permite desplegar servicios web Docker y sitios estáticos, con HTTPS automático y despliegue continuo desde GitHub.

4.3 Control de versiones

Durante todo el desarrollo de PlateUp se utilizó Git como sistema de control de versiones distribuido, con el repositorio remoto alojado en GitHub. El flujo de trabajo siguió un modelo de commits frecuentes y descriptivos en la rama principal (master), lo que permite seguir la evolución del proyecto commit a commit y revertir cambios si fuera necesario.

El historial completo de commits en Git proporciona trazabilidad total del desarrollo, permitiendo ver exactamente qué cambios se hicieron en cada fase del proyecto. El repositorio está disponible en: <https://github.com/XabierLDGA/PlateUp.git>

4.4 Seguridad de la aplicación

La seguridad de PlateUp se implementa en múltiples capas, garantizando que solo los usuarios autorizados puedan realizar acciones que requieren autenticación.

Autenticación con JWT

Al registrarse o iniciar sesión correctamente, el servidor genera un JWT firmado con una clave secreta HMAC-SHA256 configurada en application.properties. El token tiene una validez de 30 días (2.592.000.000 milisegundos) y contiene como claims el username, el userId y el email del usuario.

Filtro JWT en Spring Security

La clase JwtAuthenticationFilter extiende OncePerRequestFilter e intercepta todas las peticiones entrantes. Extrae el token de la cabecera Authorization, lo valida usando JwtService (que verifica la firma y la expiración), obtiene el username del subject del token, carga el usuario de la base de datos y establece el contexto de seguridad de Spring Security para que los controladores puedan identificar al usuario autenticado mediante @AuthenticationPrincipal.

Política de acceso a endpoints

La clase SecurityConfig configura la política de acceso con los siguientes criterios: los endpoints /api/users/login y /api/users/register son públicos. Los endpoints GET de /api/** son públicos (cualquiera puede leer recetas). Los endpoints de escritura (POST, PUT, DELETE) requieren autenticación. Los endpoints de /api/admin/** requieren autenticación. Las

peticiones OPTIONS se permiten sin restricción para el correcto funcionamiento del CORS preflight.

Cifrado de contraseñas

Las contraseñas de los usuarios se cifran con BCrypt antes de almacenarse en la base de datos. BCrypt incorpora un salt aleatorio en cada hash, lo que garantiza que dos contraseñas idénticas generan hashes distintos y hace prácticamente imposible los ataques de diccionario o rainbow table. El bean PasswordEncoder de Spring Security se configura con BCryptPasswordEncoder.

4.5 Herramientas utilizadas durante el desarrollo

Además del stack tecnológico principal, se utilizaron diversas herramientas de apoyo para el diseño, la documentación y las pruebas. Para diseño y modelado: draw.io (diagrams.net) para el diagrama entidad-relación y diagramas de arquitectura, Figma para prototipos de baja y media fidelidad en la fase inicial y Mermaid AI para diagramas exportables a imagen.

Como entornos de desarrollo: IntelliJ IDEA para el backend Java (refactorización, depuración, Maven), Visual Studio Code para el frontend Vue.js (con extensiones Volar, Tailwind CSS IntelliSense y ESLint) y Android Studio para la compilación del APK generado con Capacitor. Para pruebas y base de datos: Swagger UI para validar todos los endpoints de la API REST y Docker Desktop para levantar el entorno completo (backend, frontend y MySQL) con un solo comando docker-compose up.

Como asistentes de IA se emplearon ChatGPT Pro (consulta técnica sobre Spring Security, JPA/Hibernate y patrones Vue.js, así como diagnóstico de errores) y Claude Code de Anthropic (directamente en el terminal del backend para refactorizaciones, generación de DTOs y resolución de errores complejos). Para control de versiones se usó Git, con commits frecuentes y descriptivos, y GitHub como repositorio remoto integrado con Railway para despliegue continuo: cada push a master desencadena automáticamente un nuevo despliegue.

5. Análisis del sistema

5.1 Actores del sistema

Usuario no autenticado

Usuario que accede a la aplicación sin haber iniciado sesión. Su acceso se limita a las pantallas de registro e inicio de sesión (AuthView). El router de Vue.js implementa una guarda de navegación que redirige automáticamente al login cualquier intento de acceder a rutas protegidas.

Usuario autenticado

Usuario principal de la aplicación. Tras iniciar sesión y recibir el token JWT, tiene acceso completo a todas las funcionalidades: feed, creación de recetas, interacción social, perfil y modo cocina. El sistema lo identifica en cada petición mediante el token JWT en la cabecera de autorización.

Administrador

Usuario con rol de administrador con acceso adicional al panel (/admin). El router implementa una guarda adminOnly que verifica los permisos antes de permitir el acceso. El panel muestra estadísticas globales: usuarios, recetas, likes y gráficos de distribución por categorías y evolución mensual.

5.3 Requisitos funcionales

Los requisitos funcionales definen qué debe hacer el sistema para satisfacer las necesidades de los usuarios:

- RF-01: El sistema debe permitir el registro de nuevos usuarios con username único, email único y contraseña. El sistema debe validar que no existan usuarios con el mismo username o email antes de crear la cuenta.
- RF-02: El sistema debe permitir el inicio de sesión con username o email y contraseña. Tras una autenticación correcta, el sistema debe devolver un token JWT válido durante 30 días.
- RF-03: El sistema debe proteger todos los endpoints de escritura (POST, PUT, DELETE) exigiendo un token JWT válido en la cabecera Authorization de la petición.
- RF-04: El sistema debe permitir a los usuarios autenticados crear recetas con título, descripción, imagen, categoría, tiempo, dificultad, raciones, lista de ingredientes y pasos de preparación.
- RF-05: El sistema debe mostrar un feed de recetas en la pantalla principal ordenado por fecha de publicación, con posibilidad de filtrar por categoría.
- RF-06: El sistema debe permitir dar y quitar like a cualquier receta. Cada usuario puede dar un único like por receta.
- RF-07: El sistema debe permitir seguir y dejar de seguir a cualquier otro usuario. El sistema debe registrar la fecha de inicio del seguimiento.
- RF-08: El sistema debe permitir la búsqueda de recetas y usuarios por nombre en tiempo real.

- RF-09: El sistema debe mostrar el perfil de cada usuario con sus recetas publicadas, número de seguidores, número de seguidos y logros obtenidos.
- RF-10: El sistema debe proporcionar un modo cocina guiado con navegación paso a paso y temporizadores.
- RF-11: El sistema debe permitir subir imágenes de recetas y de perfil en formatos PNG, JPEG y WebP.
- RF-12: El sistema debe proporcionar un panel de administración con estadísticas globales de la plataforma, accesible únicamente para usuarios con rol de administrador.

5.4 Requisitos no funcionales

Rendimiento

La API REST debe responder a peticiones de lectura (GET) en menos de 500 ms en condiciones normales, y las de escritura (POST/PUT/DELETE) en menos de 1 segundo. El frontend debe cargar el feed en menos de 2 segundos en 4G estándar. Las imágenes se optimizan antes de enviarse al servidor. HikariCP está configurado con timeout de 30 s y tiempo de inicialización de 60 s, adecuados para Railway.

El sistema de pool de conexiones de HikariCP está configurado con un timeout de conexión de 30 segundos y un tiempo de inicialización de 60 segundos, parámetros adecuados para el entorno de despliegue en Railway.

Usabilidad

La interfaz debe ser intuitiva y no requerir formación previa. Las acciones principales (crear receta, dar like, seguir usuario) deben estar accesibles en máximo dos toques. La barra inferior (BottomNav) está siempre visible en las pantallas principales. El diseño es responsive desde 360px (Android pequeño) hasta 1440px (escritorio). Los formularios proporcionan feedback inmediato: mensajes de error descriptivos, indicadores de carga y confirmación visual de acciones.

Los formularios deben proporcionar feedback inmediato al usuario: mensajes de error descriptivos cuando los datos son incorrectos, indicadores de carga durante las operaciones asíncronas, y confirmación visual de las acciones completadas con éxito.

Seguridad

Ningún usuario puede modificar ni eliminar contenido ajeno. Los tokens JWT se firman con clave secreta de al menos 256 bits. Las contraseñas se almacenan cifradas con BCrypt. La comunicación en producción es HTTPS. Las variables sensibles no se incluyen en el repositorio. La protección CORS evita peticiones desde dominios externos no autorizados, y los archivos subidos se validan en tipo y tamaño antes de almacenarse.

El sistema debe implementar protección CORS correctamente configurada para evitar peticiones no autorizadas desde dominios externos. Los archivos subidos al servidor deben validarse en tipo y tamaño antes de almacenarse para prevenir la subida de archivos maliciosos.

Disponibilidad

El sistema debe estar disponible al menos el 99% del tiempo durante el período de evaluación del proyecto. El despliegue en Railway con plan gratuito introduce la limitación de un límite mensual de horas de ejecución, por lo que el servidor puede dejar de responder si se agotan

los créditos del mes, lo cual es aceptable en el contexto de un proyecto académico. Para un entorno de producción real, se optaría por un plan de pago que garantice disponibilidad continua sin restricción de horas.

Mantenibilidad

El código sigue los principios SOLID y se organiza en capas bien diferenciadas (presentación, lógica de negocio, acceso a datos). El backend se documenta mediante Springdoc OpenAPI (accesible en `/swagger-ui/index.html`) y el código Vue.js sigue las convenciones oficiales con componentes SFC organizados.

Escalabilidad

La arquitectura stateless del backend (sin sesiones, con JWT) permite escalar horizontalmente añadiendo instancias sin compartir estado. La separación frontend/backend permite escalar cada componente de forma independiente. La base de datos MySQL en Railway puede ampliarse a planes superiores según crezca el volumen.

Compatibilidad

La aplicación web debe funcionar correctamente en los navegadores modernos más utilizados: Chrome 90+, Firefox 88+, Safari 14+ y Edge 90+. La aplicación Android debe funcionar en dispositivos con Android 7.0 (API level 24) o superior. La interfaz debe adaptarse correctamente a los tamaños de pantalla más comunes: 360px, 414px, 768px y 1440px de ancho.

5.5 Historias de usuario

Las historias de usuario describen las funcionalidades del sistema desde el punto de vista del usuario final, siguiendo el formato estándar: Como [rol], quiero [acción], para [beneficio]. Cada historia incluye sus criterios de aceptación que definen cuándo se considera completada.

Autenticación y gestión de cuenta

HU-01 — REGISTRO DE USUARIO

Como usuario nuevo quiero registrarme con usuario, email y contraseña para acceder a todas las funcionalidades de PlateUp. Criterios: formulario con los tres campos, email único, contraseña mínimo 8 caracteres, confirmación por email y mensaje de éxito. Tamaño: 8 puntos | Prioridad: Alta

HU-02 — INICIO DE SESIÓN

Como usuario registrado quiero iniciar sesión con username o email y contraseña para acceder a mi cuenta y contenido. Criterios: validar credenciales, mantener sesión activa, recordar usuario (opcional), mensaje de error si fallan datos. Tamaño: 5 puntos | Prioridad: Alta

HU-03 — EDICIÓN DE PERFIL

Como usuario autenticado quiero editar mi foto de perfil para personalizar mi presencia en la comunidad. Criterios: subir imagen, previsualizar antes de guardar, formatos PNG/JPG, máximo 5MB. Tamaño: 3 puntos | Prioridad: Media

Administración

HU-11 — CONSULTAR LAS ESTADÍSTICAS DE LA PLATAFORMA

Como administrador quiero consultar las estadísticas globales (usuarios, recetas, likes, comentarios y gráficos de evolución) para tomar decisiones informadas sobre el crecimiento del servicio. Criterios: dashboard con KPIs principales, gráficos de evolución (últimos 30 días), filtros por rango de fechas, exportar datos. Tamaño: 13 puntos | Prioridad: Media

6. Desarrollo técnico detallado

6.1 Metodología de trabajo

La metodología fue iterativa e incremental: el proyecto se construyó progresivamente, añadiendo funcionalidades completas en cada iteración y manteniendo siempre una versión funcional. Cada iteración seguía el ciclo: análisis, diseño técnico, implementación backend, prueba con Swagger UI, implementación frontend, integración y commit descriptivo. Git con commits frecuentes en GitHub fue fundamental para la trazabilidad y el respaldo continuo.

Cada iteración seguía el mismo ciclo: análisis de la funcionalidad a implementar, diseño de la solución técnica, implementación en el backend, prueba de los endpoints con Swagger UI, implementación en el frontend, integración y prueba end-to-end, y commit al repositorio con un mensaje descriptivo del trabajo realizado.

El control de versiones con Git fue fundamental para la gestión del desarrollo. Los commits frecuentes con mensajes claros permitieron tener un historial detallado de la evolución del proyecto, facilitar la reversión de cambios problemáticos y mantener una copia de seguridad remota siempre actualizada en GitHub.

6.2 Desarrollo del backend (Spring Boot + API REST)

Estructura del proyecto

El backend sigue la estructura estándar de paquetes de Spring Boot, con cada capa claramente separada bajo `es.ieslavereda.plateup`: `model` (entidades JPA con `@Entity` y relaciones `@OneToMany`/`@ManyToOne`/`@ManyToMany`/`@EmbeddedId`), `repository` (interfaces que extienden `JpaRepository` con métodos derivados del nombre), `service` (clases `@Service` con la lógica de negocio, como `AchievementUnlockService` y `FileStorageService`), `controller` (clases `@RestController` que gestionan las peticiones HTTP), `security` (`JwtService` y `JwtAuthenticationFilter`), `config` (`SecurityConfig`, `WebConfig`, `AchievementSyncRunner`), `dto` (`LoginRequest`, `RegisterRequest`, `AuthResponse`) y `exception` (`GlobalExceptionHandler` con `@ControllerAdvice` y excepciones personalizadas como `ResourceNotFoundException` y `AuthenticationRequiredException`).

Controladores REST implementados

PlateUp cuenta con 18 controladores REST que cubren todas las entidades del sistema: `UserController` (registro, login, perfil, búsqueda, seguidores), `RecipeController` (CRUD de recetas, listado, filtrado, búsqueda), `RecipeStepController` y `RecipeIngredientController` (gestión de pasos e ingredientes), `IngredientController` y `UtensilController` (catálogos), `LikeController` y `FollowController` (interacciones sociales), `AchievementController`, `UserAchievementController`, `ChallengeController` y `UserChallengeController` (gamificación), `CollectionController` y `CollectionRecipeController` (colecciones personales), `CommentController`, `CookedRecipeController` (registro de recetas cocinadas), `UploadController` (subida de imágenes) y `AdminController` (estadísticas globales).

Cada controlador responde con `ResponseEntity` y los códigos HTTP apropiados (200, 201, 400, 401, 404). La lista detallada de endpoints con sus métodos HTTP, rutas y cuerpos de petición se encuentra en el Anexo II.

Documentación automática con Springdoc OpenAPI

El proyecto incluye springdoc-openapi-starter-webmvc-ui, que genera automáticamente la documentación interactiva en formato Swagger UI, disponible en <http://localhost:8080/swagger-ui/index.html>. Este endpoint está configurado como público en Spring Security para facilitar el acceso durante el desarrollo.

6.3 Desarrollo del frontend (Vue.js 3)

Estructura del proyecto frontend

El proyecto frontend se organiza dentro de src/ con las siguientes carpetas: views/ contiene las 11 vistas principales como componentes Vue SFC (HomeView, ExploreView, CreateView, ProfileView, etc.); components/ aloja componentes reutilizables como BottomNav.vue y RecipeCard.vue; router/index.js configura todas las rutas, guards de navegación y redirecciones; stores/authStore.js gestiona el estado de autenticación con Pinia (token JWT, datos del usuario, métodos initialize/login/logout); services/ contiene un archivo por entidad (userService.js, recipeService.js, likeService.js, etc.) que encapsula las llamadas Axios a la API; utils/ aloja funciones de utilidad (recipeCategories.js define las 14 categorías, media.js, userAvatar.js, recipeImage.js); y assets/main.css define los estilos globales con las directivas de Tailwind CSS v4.

Interceptor de Axios

El archivo src/services/api.js configura la instancia global de Axios con la URL base (VITE_API_URL) e implementa un interceptor que añade automáticamente la cabecera Authorization: Bearer {token} a todas las peticiones, leyendo el token de localStorage. Ningún servicio necesita gestionar la autenticación manualmente.

Guard de navegación en Vue Router

El archivo router/index.js implementa un beforeEach global: si la ruta requiere autenticación y el usuario no la tiene, lo redirige al login; si la ruta tiene meta adminOnly y el usuario no es administrador, lo redirige a la pantalla principal. La raíz / redirige al perfil si está autenticado o al login si no lo está.

6.4 Aplicación móvil Android con Capacitor

La versión Android de PlateUp se desarrolló utilizando Capacitor 8, que transforma la aplicación Vue.js en una aplicación Android instalable. El proceso de desarrollo siguió estos pasos:

- Configuración de Capacitor en el proyecto Vue.js mediante el archivo capacitor.config.json, que define el identificador del paquete (com.plateup.app), el nombre de la aplicación y la URL del servidor web local para el modo de desarrollo.
- Construcción del proyecto Vue.js con el comando npm run build, que genera los archivos estáticos optimizados en el directorio dist/.
- Sincronización del directorio dist/ con el proyecto Android mediante el comando npx cap sync android, que copia los archivos web al directorio android/app/src/main/assets/public/.
- Apertura del proyecto en Android Studio con npx cap open android para compilar el APK y ejecutarlo en un dispositivo o emulador Android.

- Configuración del AndroidManifest.xml con los permisos necesarios: acceso a internet (INTERNET), acceso al almacenamiento de archivos (READ_EXTERNAL_STORAGE, WRITE_EXTERNAL_STORAGE).

El archivo capacitor.config.json configura el comportamiento de la aplicación en Android, incluyendo la política de uso de cámara para la subida de imágenes de recetas y avatares desde la galería.

6.5 Sistema de subida de imágenes

PlateUp implementa un sistema de subida de imágenes gestionado por `FileStorageService` en el backend y `UploadController` que expone el endpoint `POST /api/upload`. El frontend utiliza el componente de input tipo file para seleccionar la imagen, la previsualiza localmente con `URL.createObjectURL()`, y la envía al servidor en formato multipart/form-data mediante `Axios`.

El servidor valida el tipo MIME y tamaño (máximo 5 MB), genera un nombre único basado en UUID y lo almacena en el directorio `uploads/` (variable `APP_UPLOAD_DIR`). La URL pública del archivo (`/uploads/{nombre}`) se devuelve al frontend, que la guarda en `image_url` de la receta o `avatar_url` del usuario.

6.6 Sistema de gamificación

El sistema de gamificación se basa en logros (Achievements) y retos (Challenges). Los logros son hitos permanentes que se desbloquean al cumplir condiciones (publicar la primera receta, alcanzar 10 seguidores, cocinar 5 recetas, etc.).

Los retos son desafíos temporales con fecha de inicio y fin, objetivo y recompensa. La estructura está completamente implementada (tablas `Challenges` y `UserChallenges`, endpoints REST funcionales). La automatización del seguimiento del progreso está diseñada para versión futura: requeriría un `ChallengeProgressService` integrado en los puntos donde el usuario realiza acciones cuantificables, actualizando el progreso y marcando retos completados. `AchievementSyncRunner` es un `CommandLineRunner` que sincroniza los logros definidos en el código con la base de datos al arrancar el servidor. `AchievementUnlockService` evalúa las condiciones de desbloqueo y se invoca desde `RecipeController`, `FollowController` y `CookedRecipeController`.

6.7 Pruebas realizadas

Las pruebas del sistema se realizaron de forma manual y sistemática en dos entornos: local (con el servidor Spring Boot corriendo en `localhost:8080` y la base de datos MySQL local) y producción (con el backend en Railway y la base de datos en Railway).

Pruebas de la API REST

Cada endpoint se probó con Swagger UI, verificando códigos HTTP correctos (200, 201, 400, 401, 404), cuerpos JSON esperados y el comportamiento de la autenticación JWT (acceso con token válido, rechazo con token inválido/expirado, rechazo sin token en endpoints protegidos).

Pruebas de la aplicación web

Se probaron manualmente todos los flujos de usuario: registro, inicio de sesión, creación de receta con ingredientes y pasos, subida de imagen, like, follow, navegación entre vistas, edición de perfil y modo cocina. Verificado en Chrome, Firefox y Edge.

Pruebas de la aplicación Android

La versión Android se probó en un dispositivo físico Android real, verificando que la interfaz se visualiza correctamente en pantalla móvil, que la navegación funciona con gestos táctiles, que la subida de imágenes desde la galería funciona correctamente, y que la comunicación con la API de producción en Railway funciona sin problemas.

Pruebas con usuarios reales

Adicionalmente, la aplicación se presentó a usuarios reales (familiares y conocidos) como testers independientes. Este feedback identificó errores de usabilidad, flujos no intuitivos y casos límite no considerados. Las observaciones de usuarios sin contexto técnico revelaron asunciones implícitas en el diseño y cada sesión generó correcciones que mejoraron la experiencia final.

6.8 Despliegue en producción (Railway)

El despliegue en producción de PlateUp se realizó utilizando el servicio de cloud de Railway.

Base de datos MySQL gestionada

Railway permite gestionar servicios de aplicación y bases de datos en un único entorno. Se eligió para centralizar el despliegue de backend y base de datos, por su interfaz sencilla para MySQL y por proporcionar conexión SSL cifrada y variables de entorno seguras. Incluye también una interfaz web para explorar los datos directamente desde el navegador.

Se crea un servicio MySQL en Railway, se obtiene la cadena JDBC (host, puerto, base, usuario y contraseña) y se configura como variable de entorno.

Backend (API REST)

El backend se despliega en Railway como Web Service Docker usando el Dockerfile incluido en el directorio plateup/. El archivo railway.toml en la raíz define la configuración completa: nombre, runtime Docker, ruta del Dockerfile, contexto de construcción y variables de entorno.

Frontend (Sitio estático)

El frontend se despliega como sitio web estático en Railway, accesible desde cualquier dispositivo con conexión a internet. Los archivos generados por npm run build se sincronizan con el proyecto Android mediante npx cap sync android y se empaquetan desde Android Studio. La variable VITE_API_URL apunta a la URL de producción del backend.

Docker Compose para desarrollo local

Para el desarrollo local, el repositorio incluye un docker-compose.yml que levanta tres contenedores: MySQL 8.0 con charset utf8mb4, el backend Spring Boot y el frontend Vue.js servido con Nginx.

7. Base de datos

7.1 Diseño conceptual

El diseño parte de un análisis exhaustivo de las funcionalidades de la aplicación. El modelo debe soportar la gestión de usuarios con perfil completo, recetas con todos sus componentes (pasos, ingredientes, utensilios), interacciones sociales (likes, follows), colecciones, gamificación (logros, retos), modo cocina (recetas cocinadas) y funciones de administración (reportes, auditoría). Se adoptó un modelo relacional normalizado con tablas intermedias para relaciones muchos a muchos, y charset utf8mb4 (collation utf8mb4_unicode_ci) para soporte completo de Unicode incluyendo emojis.

Se adoptó un modelo relacional normalizado para eliminar redundancias y garantizar la integridad de los datos. Las relaciones muchos a muchos (como Usuarios-Recetas en likes, o Recetas-Ingredientes) se implementan mediante tablas intermedias con claves compuestas.

El charset elegido para toda la base de datos es utf8mb4 con collation utf8mb4_unicode_ci, que soporta el rango completo de caracteres Unicode incluyendo emojis, necesarios para los textos de recetas y los iconos de logros.

7.2 Descripción de las entidades

El modelo de PlateUp se compone de las siguientes entidades principales: Users (datos del usuario, credenciales, avatar y visibilidad por defecto), Recipes (recetas publicadas con título, descripción, imagen, raciones, tiempo, dificultad), RecipeSteps (pasos numerados con texto y temporizador opcional), Ingredients (catálogo global de ingredientes), RecipeIngredients (tabla intermedia con cantidad, unidad y notas), Utensils y RecipeUtensils (catálogo de utensilios y su relación con recetas), Follows (relaciones de seguimiento entre usuarios), Likes (likes con clave compuesta user_id + recipe_id), Collections y CollectionRecipes (colecciones personales de recetas), Achievements y UserAchievements (logros del sistema y los obtenidos por cada usuario), Challenges y UserChallenges (retos y progreso individual), CookedRecipes (registro de las veces que un usuario ha cocinado una receta con tiempo transcurrido), Comments (comentarios sobre recetas), y otras tablas auxiliares definidas para uso futuro (Media, Notifications, Reports, AuditLogs). Todas las relaciones muchos a muchos se implementan con tablas intermedias y claves compuestas, y las claves foráneas utilizan ON DELETE CASCADE para garantizar la integridad referencial. El detalle completo de campos y tipos se especifica en la siguiente tabla 7.3.

7.3 Paso a tablas (modelo relacional completo)

Tabla	Definición de campos
USERS	id BIGINT PK AI, username VARCHAR(50) UNIQUE NOT NULL, email VARCHAR(100) UNIQUE NOT NULL, password_hash VARCHAR(255) NOT NULL, display_name VARCHAR(100), bio TEXT, avatar_url VARCHAR(255), visibility_default ENUM('private','followers','public') DEFAULT 'public', created_at DATETIME, updated_at DATETIME

RECIPES	id BIGINT PK AI, user_id BIGINT FK→Users ON DELETE CASCADE, title VARCHAR(150) NOT NULL, description TEXT, image_url VARCHAR(500), servings INT, total_minutes INT, difficulty ENUM('easy','medium','hard'), created_at DATETIME, updated_at DATETIME
RECIPESTEPS	id BIGINT PK AI, recipe_id BIGINT FK→Recipes ON DELETE CASCADE, order_index INT NOT NULL, text TEXT NOT NULL, timer_seconds INT, media_url VARCHAR(255)
INGREDIENTS	id BIGINT PK AI, name VARCHAR(100) UNIQUE NOT NULL, unit_default VARCHAR(20)
RECIPEINGREDIENTS	id BIGINT PK AI, recipe_id BIGINT FK→Recipes ON DELETE CASCADE, ingredient_id BIGINT FK→Ingredients ON DELETE CASCADE, quantity_decimal DECIMAL(10,2), unit VARCHAR(20), notes TEXT
UTENSILS	id BIGINT PK AI, name VARCHAR(100) UNIQUE NOT NULL
RECIPEUTENSILS	recipe_id BIGINT FK→Recipes ON DELETE CASCADE, utensil_id BIGINT FK→Utensils ON DELETE CASCADE, PK(recipe_id, utensil_id)
FOLLOWS	follower_id BIGINT FK→Users ON DELETE CASCADE, followed_id BIGINT FK→Users ON DELETE CASCADE, status ENUM('pending','active') DEFAULT 'active', created_at DATETIME, PK(follower_id, followed_id)
LIKES	user_id BIGINT FK→Users ON DELETE CASCADE, recipe_id BIGINT FK→Recipes ON DELETE CASCADE, created_at DATETIME, PK(user_id, recipe_id)
COLLECTIONS	id BIGINT PK AI, user_id BIGINT FK→Users ON DELETE CASCADE, name VARCHAR(100) NOT NULL, is_public TINYINT(1) DEFAULT 1, created_at DATETIME
COLLECTIONRECIPES	collection_id BIGINT FK→Collections ON DELETE CASCADE, recipe_id BIGINT FK→Recipes ON DELETE CASCADE, order_index INT, PK(collection_id, recipe_id)
ACHIEVEMENTS	id BIGINT PK AI, name VARCHAR(100) NOT NULL, description TEXT, icon_url VARCHAR(255), category VARCHAR(50), points INT DEFAULT 0, created_at DATETIME
USERACHIEVEMENTS	user_id BIGINT FK→Users ON DELETE CASCADE, achievement_id BIGINT FK→Achievements ON DELETE CASCADE, earned_at DATETIME, level INT DEFAULT 1, PK(user_id, achievement_id)
CHALLENGES	id BIGINT PK AI, name VARCHAR(150) NOT NULL, description TEXT, start_date DATETIME, end_date DATETIME, goal INT, reward_points INT DEFAULT 0
USERCHALLENGES	user_id BIGINT FK→Users ON DELETE CASCADE, challenge_id BIGINT FK→Challenges ON DELETE CASCADE, progress INT DEFAULT 0, status ENUM('in_progress','completed','abandoned') DEFAULT

	'in_progress', started_at DATETIME, finished_at DATETIME, PK(user_id, challenge_id)
COMMENTS	id BIGINT PK AI, user_id BIGINT FK→Users ON DELETE CASCADE, recipe_id BIGINT FK→Recipes ON DELETE CASCADE, text TEXT NOT NULL, created_at DATETIME
COOKEDRECIPES	id BIGINT PK AI, user_id BIGINT FK→Users ON DELETE CASCADE, recipe_id BIGINT FK→Recipes ON DELETE CASCADE, cooked_at DATETIME, elapsed_seconds INT

8. Diseño visual del proyecto (UX/UI)

8.1 Filosofía de diseño y decisiones generales

El diseño de PlateUp sigue una filosofía mobile-first, partiendo de la interfaz más pequeña (dispositivo Android de 360-414px) y escalando hacia el entorno de escritorio. Todas las decisiones de diseño están orientadas a maximizar la usabilidad en pantallas táctiles pequeñas: elementos interactivos con área de toque generosa (mínimo 44x44px), navegación accesible con el pulgar, cards con bordes redondeados y sin esquinas afiladas, y uso generoso del espacio en blanco para evitar la sensación de aglomeración.

La paleta de colores se basa en un color principal rojizo-naranja (#f45b3f) para las acciones primarias y el contenido destacado, fondos muy claros (#f6f3ee) para reducir la fatiga visual, tarjetas blancas con sombra muy suave para separar el contenido, y acentos naranjas suaves (orange-50) para los estados de hover y las secciones de información secundaria. Esta paleta transmite calidez y energía, coherente con el ámbito de la cocina.

La tipografía utiliza la fuente del sistema operativo (system-ui) con pesos semibold y bold para títulos y elementos de navegación, y regular para el texto de contenido. Los tamaños varían entre 12px (etiquetas secundarias), 14-16px (cuerpo de texto) y 24-32px (títulos de sección).

La navegación principal se organiza en una barra inferior fija (BottomNav) con cuatro iconos: Home (feed principal), Explore (descubrimiento), Create (nueva receta) y Profile (perfil propio). Esta barra es persistente en todas las pantallas principales y ofrece acceso inmediato a cualquier sección desde cualquier punto de la aplicación.

8.2 AuthView — Pantalla de autenticación

Punto de entrada para usuarios no autenticados. Diseño limpio con el logotipo de PlateUp y un formulario que alterna entre inicio de sesión (username y contraseña) y registro (username, email y contraseña) sin recargar la página. Incluye validación del lado del cliente y mensajes de error descriptivos. Tras un inicio de sesión exitoso, el token JWT se almacena en el store de Pinia y el router redirige al perfil; tras un registro, el usuario queda autenticado directamente.

8.3 HomeView — Feed principal

Núcleo de la experiencia social. Se organiza en encabezado con saludo, título y avatar (acceso rápido al perfil); fila horizontal con scroll de los 14 filtros de categoría más la opción 'All', con el filtro activo destacado en color principal; tarjeta Cooking Journey con gradiente naranja que muestra la racha de días, recetas publicadas, medallas y puntos; y feed de recetas ordenadas por fecha mediante el componente RecipeCard, con likes accesibles directamente desde la tarjeta sin entrar al detalle.

8.4 ExploreView — Exploración y búsqueda

Pantalla orientada al descubrimiento. Incluye una barra de búsqueda reactiva que filtra recetas y usuarios en tiempo real consultando la API, una sección horizontal 'People to discover' con tarjetas de usuario que muestran avatar, nombre y botón Follow/Following con

cambio visual inmediato, y un grid de dos columnas estilo Instagram con las recetas de la plataforma priorizando la imagen como elemento principal de atracción.

8.5 RecipeDetailView — Detalle de receta

Una de las pantallas más ricas en contenido. Hero superior con imagen panorámica, gradiente oscuro y superposición de título, descripción y botón de like con contador; fila de metadatos (dificultad, tiempo, raciones, categoría); ficha del autor con avatar y acceso a su perfil; tarjeta de ingredientes con checkbox para marcar los preparados; tarjeta de pasos numerados; y botón destacado 'Start Cooking' que lanza el modo cocina guiado.

8.6 CookingView — Modo cocina guiado

Vista optimizada para usar mientras se cocina, con elementos grandes y simplificados pensados para manos mojadas o enharinadas. Temporizador global en formato MM:SS con botones para pausar y reanudar, barra de progreso horizontal que indica el avance entre pasos, presentación de los pasos uno a uno a pantalla completa, y botón 'Finish Cooking' en el último paso que registra la receta como cocinada junto con el tiempo transcurrido en la tabla CookedRecipes.

8.7 CreateView — Creación de receta

Formulario más complejo de la aplicación, organizado en secciones con validación en tiempo real: información básica (título y descripción), área de previsualización de imagen con botones para elegir o eliminar archivo, metadatos (categoría, dificultad, tiempo, raciones), sección dinámica de ingredientes (cada uno con nombre, cantidad y unidad, editables y eliminables) y sección dinámica de pasos numerados. El botón 'Publish' envía todos los datos al backend mediante FormData.

8.8 ProfileView — Perfil propio

Espacio personal del usuario. Tarjeta superior con avatar circular, display name, username, biografía y tres métricas (recetas, seguidores y seguidos, con los dos últimos interactivos para abrir las listas). Dos tabs alternan entre 'My Recipe Book' (recetas publicadas con opción exclusiva de eliminar, más recetas cocinadas recientemente) y 'Achievements' (logros obtenidos con icono, nombre y puntos). En la parte inferior, menú Settings con 'Edit Profile' y 'Log Out'.

8.9 UserProfileView — Perfil de otros usuarios

Estructura idéntica al perfil propio pero sin opciones de edición ni eliminación de recetas. Incluye botón Follow/Following y las listas de seguidores y seguidos siguen siendo accesibles. Al tocar una receta del usuario se navega a su detalle.

8.10 EditProfileView — Edición del perfil

Permite actualizar los datos personales. En la parte superior, una tarjeta destacada de fondo naranja suave muestra el avatar actual con opciones 'Change profile photo' y 'Remove' que abren el selector de archivos del dispositivo o cancelan la selección. Los cambios de avatar se guardan inmediatamente al confirmar.

8.11 ConnectionsView — Seguidores y seguidos

Muestra dos listas de usuarios (seguidores y seguidos) con un selector superior para alternar entre ellas. Cada usuario aparece con avatar, nombre y username; al tocarlo se navega a su perfil. Esta pantalla puede accederse tanto desde el perfil propio como desde el perfil de otro usuario.

8.12 AdminDashboardView — Panel de administración

Vista exclusiva para usuarios con rol de administrador (ruta /admin con guard adminOnly). Muestra los KPIs globales en cuadrícula 2x2 (usuarios, recetas, likes y recetas cocinadas), un gráfico de donut con la distribución por categoría, un gráfico de barras con la evolución mensual de nuevos usuarios en los últimos 6 meses y otro con el ranking de creadores más prolíficos. Todos los gráficos se renderizan con Chart.js a partir del endpoint del AdminController.

9. Mejoras a largo plazo

Aunque PlateUp en su versión actual presenta una plataforma funcional, completa y bien estructurada, existen múltiples líneas de mejora y expansión que se contemplan para versiones futuras del proyecto. Estas mejoras se han priorizado en función de su impacto potencial en la experiencia del usuario y la viabilidad técnica:

9.1 Sistema de notificaciones en tiempo real

Implementación de notificaciones push para informar en tiempo real de likes, follows, comentarios y retos disponibles. Requeriría WebSockets en el backend (Spring WebSocket o Server-Sent Events) e integración con las notificaciones nativas de Android mediante plugins de Capacitor.

9.2 Recomendaciones personalizadas con IA

Sistema de recomendación inteligente que analizaría el historial de interacciones de cada usuario (recetas cocinadas, categorías preferidas, usuarios seguidos, likes) para sugerir contenido relevante en el feed y en Explore. Podría implementarse con filtrado colaborativo o modelos de machine learning; la tabla CookedRecipes ya almacena los datos necesarios.

9.3 Integración con compra de ingredientes

Añadir ingredientes de una receta directamente a la cesta de supermercados online (Mercadona, Carrefour, Amazon Fresh). Generaría ingresos por comisión de afiliado y requeriría integración con las APIs de los supermercados o con agregadores de precios.

9.4 Versión iOS con Capacitor

Como el proyecto ya usa Capacitor, la extensión a iOS es relativamente sencilla: el mismo proyecto puede compilarse para ambas plataformas. Requeriría cuenta de Apple Developer Program (99\$/año), un Mac con Xcode y pruebas en dispositivos iOS.

9.5 Mensajería directa entre usuarios

Sistema de chat privado entre usuarios para intercambiar recetas, fotos de platos y consejos. Requeriría WebSockets, modelo de datos adicional (Messages, Conversations) y una vista de chat en el frontend.

9.6 Mejoras en el sistema de gamificación

Ampliar el sistema con temporadas de desafíos con clasificaciones globales, eventos especiales para festividades gastronómicas, rankings semanales y niveles de experiencia en el perfil.

9.7 Internacionalización (i18n)

Soporte para múltiples idiomas en la interfaz mediante vue-i18n en el frontend y configuraciones de locale en el backend. Prioridad: español, inglés e italiano, ampliable a otros idiomas europeos.

9.8 Almacenamiento de imágenes en CDN

Migrar el almacenamiento de imágenes a un CDN especializado (Cloudinary, AWS S3 + CloudFront), mejorando los tiempos de carga gracias a la distribución geográfica, reduciendo la carga del servidor y eliminando la pérdida de imágenes cuando Railway se reinicia (el almacenamiento local del plan gratuito es efímero).

9.9 Sistema de reportes y moderación

Activar el sistema de reportes cuya tabla Reports ya está definida. Los usuarios podrían reportar contenido inapropiado y el administrador tendría una cola de revisión con opciones para aprobar, rechazar o eliminar. La moderación podría automatizarse parcialmente con un agente de IA.

9.10 Perfiles públicos y privados con sistema de solicitudes de seguimiento

Configuración de privacidad en perfiles: públicos (visibles para todos) y privados (requieren solicitud de seguimiento aceptada). Incluiría notificaciones de solicitudes y gestor de seguimientos, con una tabla de solicitudes y filtrado en ExploreView.

9.11 Implementación de refresh tokens

Sustitución del esquema actual de token JWT único con expiración de 30 días por un sistema de doble token: un access token de corta duración (15 minutos) para autenticar cada petición y un refresh token de larga duración (30 días) almacenado de forma segura para renovar el access token sin necesidad de reintroducir credenciales. Reduciría la ventana de exposición ante un robo de token, permitiría revocar sesiones desde el panel de administración mediante una tabla RefreshTokens y mejoraría la experiencia del usuario al mantener la sesión activa de forma transparente. Requeriría un nuevo endpoint POST /api/auth/refresh, un interceptor en Axios que renueve automáticamente el access token cuando reciba un 401, y la rotación del refresh token en cada renovación para detectar usos indebidos.

10. Conclusión

El desarrollo de PlateUp ha supuesto un ejercicio integrador de prácticamente todos los conocimientos del ciclo DAM. Desde el diseño del modelo de datos hasta el despliegue en producción, cada componente ha requerido aplicar de forma práctica conceptos trabajados en los distintos módulos.

El resultado es una aplicación funcional, desplegada en producción y accesible desde cualquier dispositivo, que resuelve un problema real: la inexistencia de una plataforma específica de cocina con la componente social de las redes generalistas.

Las decisiones de arquitectura han demostrado ser acertadas: la separación entre frontend y backend ha facilitado el desarrollo paralelo y la prueba independiente; la autenticación stateless con JWT ha sido conveniente para el despliegue distribuido; Spring Data JPA ha reducido el código repetitivo en persistencia; y Vue.js ha demostrado ser excelente por su curva de aprendizaje y reactividad intuitiva.

El despliegue en Railway ha permitido trabajar con herramientas DevOps reales: contenedores Docker, variables de entorno para configuración de producción, bases de datos gestionadas en la nube con SSL y pipelines de despliegue automático desde GitHub.

El desarrollo ha confirmado que la metodología incremental es la más adecuada para proyectos de esta envergadura: mantener siempre una versión funcional, avanzar en pequeños pasos bien definidos y usar el control de versiones de forma disciplinada son hábitos que marcan la diferencia.

En definitiva, PlateUp es mucho más que un proyecto de fin de ciclo: es una demostración real de competencia técnica, capacidad de planificación y visión de producto. Un proyecto que, con los recursos adecuados, podría evolucionar hacia un producto comercial viable.

11. Bibliografía y fuentes consultadas

A continuación se relacionan las principales fuentes consultadas durante el desarrollo de PlateUp, agrupadas por categoría. Se incluye documentación oficial de las tecnologías empleadas, asistentes de inteligencia artificial utilizados como apoyo durante el desarrollo, y recursos audiovisuales en inglés (principalmente tutoriales en YouTube) que ayudaron a comprender en profundidad conceptos clave de seguridad como JWT y BCrypt.

Documentación oficial del stack tecnológico

- Spring Boot — Documentación oficial: <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/>
- Spring Security — Reference: <https://docs.spring.io/spring-security/reference/>
- Spring Data JPA — Reference: <https://docs.spring.io/spring-data/jpa/reference/>
- Hibernate ORM — Documentación oficial: <https://hibernate.org/orm/documentation/>
- Vue.js 3 — Guía oficial: <https://vuejs.org/guide/introduction.html>
- Vue Router — Documentación oficial: <https://router.vuejs.org/>
- Pinia — Gestor de estado oficial para Vue: <https://pinia.vuejs.org/>
- Tailwind CSS v4 — Documentación oficial: <https://tailwindcss.com/docs>
- Vite — Guía oficial: <https://vitejs.dev/guide/>
- Axios — Documentación oficial: <https://axios-http.com/docs/intro>
- Capacitor — Documentación oficial (Ionic): <https://capacitorjs.com/docs>
- MySQL 8 — Reference Manual: <https://dev.mysql.com/doc/refman/8.0/en/>
- Docker — Documentación oficial: <https://docs.docker.com/>
- Railway — Documentación oficial: <https://docs.railway.app/>
- Springdoc OpenAPI — Documentación: <https://springdoc.org/>

Asistentes de inteligencia artificial

- OpenAI - ChatGPT Pro (modelo GPT). Empleado como asistente de consulta técnica para resolver dudas concretas sobre Spring Security, comportamientos de JPA/Hibernate, patrones de diseño en Vue.js 3 y diagnóstico de errores de compilación. <https://chat.openai.com/>
- Anthropic - Claude (claude.ai) y Claude Code. Empleado en el terminal del backend para refactorizaciones complejas, generación de DTOs y resolución de errores. También utilizado para revisar y compactar la presente memoria técnica. <https://www.anthropic.com/claude>
- Google - Gemini (versión gratuita). Herramienta complementaria para expandir búsquedas generales en Google, análisis de documentación y aclaraciones conceptuales sobre frameworks y patrones de arquitectura. <https://gemini.google.com/>

12. Anexos

Anexo I. Estructura completa del repositorio

El repositorio de PlateUp en GitHub (<https://github.com/XabierLDGA/PlateUp.git>) tiene la siguiente estructura de directorios:

Directorio / Archivo	Contenido
plateup/	Proyecto backend Java Spring Boot. Contiene pom.xml, Dockerfile, Dockerfile de producción y el directorio src/main/java con todos los paquetes del backend.
plateup/src/main/java/es/ieslavereda/plateup/	Paquete raíz del backend con los subpaquetes: config, controller, dto, exception, model, repository, security, service.
plateup/src/main/resources/	application.properties con la configuración del servidor (datasource, JWT, uploads, HikariCP).
Vue/plateup-frontend/	Proyecto frontend Vue.js 3 con Vite. Contiene package.json, vite.config.js, capacitor.config.json, nginx.conf y Dockerfile.
Vue/plateup-frontend/src/	Código fuente del frontend: views/, components/, router/, stores/, services/, utils/ y assets/.
Vue/plateup-frontend/android/	Proyecto Android generado por Capacitor con AndroidManifest.xml, build.gradle y carpetas de recursos de la app.
db/	Scripts SQL del proyecto: Crear_BaseDatos_Prueba.sql (DDL completo), Inserts_básicos.sql (datos de prueba), Pruebas.sql (script combinado), Limpiar_tablas.sql, Selects_Prueba.sql y Pruebas_aiven.sql.
docker-compose.yml	Orquestación completa de los tres contenedores (MySQL, backend, frontend) para desarrollo local.
railway.toml	Configuración de despliegue en Railway: dos servicios (plateup-backend como Docker, plateup-frontend como Static Site) con sus variables de entorno.
assets/	Recursos gráficos del proyecto: LogoApp.png, LogoPNG.png, LogoPDF.pdf y splash.png para la app Android.
docs/	Documentación del proyecto: PlateUp.pdf, Propuesta inicial, estructura de la memoria, notas de diseño y diagramas entidad-relación en formato PNG y SVG.

Anexo II. Endpoints completos de la API REST

A continuación se listan todos los endpoints de la API REST de PlateUp, con el método HTTP, la ruta y una descripción breve:

Endpoint	Descripción
POST /api/users/register	Registro de nuevo usuario. Body: { username, email, password }. Devuelve 201 con AuthResponse (token JWT + datos usuario).
POST /api/users/login	Inicio de sesión. Body: { username, password }. Devuelve 200 con AuthResponse.
GET /api/users/{id}	Obtener perfil de usuario por ID. Público.
PUT /api/users/{id}	Actualizar datos de perfil. Requiere autenticación y ser el propietario.
GET /api/users/search?q={term}	Buscar usuarios por username o displayName. Público.
GET /api/recipes	Listar todas las recetas ordenadas por fecha. Público.
POST /api/recipes	Crear nueva receta. Requiere autenticación. Body con todos los campos de la receta.
GET /api/recipes/{id}	Obtener detalle completo de una receta. Público.
PUT /api/recipes/{id}	Actualizar receta. Requiere autenticación y ser el propietario.
DELETE /api/recipes/{id}	Eliminar receta. Requiere autenticación y ser el propietario.
GET /api/recipes/user/{userId}	Listar recetas de un usuario. Público.
GET /api/recipe-steps/recipe/{recipeId}	Listar pasos de una receta. Público.
POST /api/recipe-steps	Crear paso de receta. Requiere autenticación.
PUT /api/recipe-steps/{id}	Actualizar paso. Requiere autenticación.
DELETE /api/recipe-steps/{id}	Eliminar paso. Requiere autenticación.
GET /api/recipe-ingredients/recipe/{recipeId}	Listar ingredientes de una receta. Público.
POST /api/recipe-ingredients	Añadir ingrediente a receta. Requiere autenticación.
DELETE /api/recipe-ingredients/{id}	Eliminar ingrediente de receta. Requiere autenticación.
POST /api/likes	Dar like a una receta. Body: { recipeId }. Requiere autenticación.

DELETE /api/likes	Quitar like. Body: { recipeld }. Requiere autenticación.
GET /api/likes/recipe/{recipeld}/count	Contar likes de una receta. Público.
GET /api/likes/user/{userId}/check/{recipeld}	Verificar si un usuario ha dado like a una receta. Público.
POST /api/follows	Seguir usuario. Body: { followedId }. Requiere autenticación.
DELETE /api/follows	Dejar de seguir. Body: { followedId }. Requiere autenticación.
GET /api/follows/{userId}/followers	Listar seguidores de un usuario. Público.
GET /api/follows/{userId}/following	Listar usuarios seguidos. Público.
GET /api/follows/check	Verificar si el usuario actual sigue a otro. Requiere autenticación.
GET /api/achievements	Listar todos los logros del sistema. Público.
GET /api/user-achievements/user/{userId}	Listar logros de un usuario. Público.
GET /api/challenges	Listar todos los retos del sistema. Público.
GET /api/collections/user/{userId}	Listar colecciones de un usuario. Público.
POST /api/collections	Crear nueva colección. Requiere autenticación.
GET /api/comments/recipe/{recipeld}	Listar comentarios de una receta. Público.
POST /api/cooked-recipes	Registrar receta cocinada. Body: { recipeld, elapsedSeconds }. Requiere autenticación.
GET /api/cooked-recipes/user/{userId}	Listar recetas cocinadas por un usuario. Requiere autenticación.
POST /api/upload	Subir imagen (multipart/form-data). Devuelve la URL pública del archivo. Requiere autenticación.
GET /api/admin/stats	Estadísticas globales de la plataforma. Requiere autenticación de administrador.

Anexo III. Variables de entorno

A continuación se listan todas las variables de entorno utilizadas por el sistema en producción:

Variable	Descripción
SPRING_DATASOURCE_URL	URL JDBC completa de conexión a MySQL en Railway, con parámetros SSL requeridos. Ejemplo: jdbc:mysql://host:port/dbname?useSSL=true
SPRING_DATASOURCE_USERNAME	Nombre de usuario de la base de datos MySQL en Railway.
SPRING_DATASOURCE_PASSWORD	Contraseña del usuario de la base de datos MySQL en Railway.
APP_JWT_SECRET	Clave secreta HMAC-SHA256 para firmar los tokens JWT. Mínimo 32 caracteres. Generada automáticamente por Railway(generateValue: true en railway.toml).
APP_UPLOAD_DIR	Directorio del servidor donde se almacenan las imágenes subidas. Valor en producción: /app/uploads.
SPRING_JPA_HIBERNATE_DDL_AUTO	Política de Hibernate para el esquema de base de datos en producción. Valor: none (no modificar el esquema automáticamente).
VITE_API_URL	URL base de la API REST, usada por el frontend en tiempo de construcción. Ejemplo: https://plateup-back.up.railway.app.

Anexo IV. Dependencias del backend (pom.xml)

El proyecto backend de PlateUp tiene las siguientes dependencias Maven declaradas en el pom.xml:

Dependencia	Versión / Descripción
spring-boot-starter-web	3.5.6 Motor web con Apache Tomcat embebido para servir la API REST.
spring-boot-starter-data-jpa	3.5.6 Integración con Spring Data JPA e Hibernate para el acceso a datos.
spring-boot-starter-security	3.5.6 Framework de seguridad para autenticación y control de acceso.
spring-boot-starter-validation	3.5.6 Validación de datos de entrada con Bean Validation (Jakarta).
jjwt-api	0.12.5 API de JSON Web Tokens para generación y validación de JWT.
jjwt-impl	0.12.5 Implementación de JWT con soporte HMAC-SHA256.
jjwt-jackson	0.12.5 Integración de JWT con Jackson para serialización JSON.
mysql-connector-j	8+ Driver JDBC para conexión con MySQL 8.
lombok	Generación automática de getters, setters, constructores y builders.
springdoc-openapi-starter-webmvc-ui	2.8.0 Generación automática de Swagger UI y documentación OpenAPI 3.

Anexo V. Dependencias del frontend (package.json)

El proyecto frontend tiene las siguientes dependencias principales declaradas en package.json:

Dependencia	Versión / Descripción
vue	3.x Framework JavaScript progresivo para la construcción de interfaces de usuario.
vue-router	4.x Enrutador oficial para Vue.js con soporte para SPA y guards de navegación.
pinia	2.x Librería de gestión de estado oficial para Vue 3.
axios	1.x Cliente HTTP para peticiones a la API REST con soporte para interceptores.
lucide-vue-next	Librería de iconos SVG optimizados para Vue 3.
@tailwindcss/vite	4.x Plugin de Vite para integrar Tailwind CSS v4 en el proceso de construcción.
tailwindcss	4.x Framework de estilos utility-first basado en Lightning CSS.
@vitejs/plugin-vue	Plugin oficial de Vite para soporte de componentes SFC de Vue.
vite	6.x Bundler y servidor de desarrollo ultrarrápido para proyectos web modernos.
@capacitor/core	8.x Core de Capacitor para la integración con plataformas nativas.
@capacitor/android	8.x Plugin de Capacitor para la plataforma Android.

Anexo VI. Modelo entidad-relación

El diagrama entidad-relación de PlateUp representa la estructura completa de la base de datos relacional, con todas las entidades del sistema y sus relaciones mediante claves foráneas. Las relaciones principales incluyen: usuarios que publican recetas, recetas con múltiples pasos e ingredientes, interacciones sociales (likes, follows), participación en logros y retos, y organización de recetas en colecciones. El diseño está normalizado en tercera forma normal.

El diagrama completo está disponible en: [Mermaid.ai – PlateUp](#)

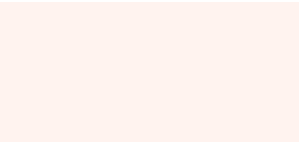
Anexo VII. Identidad visual: paleta de colores y tipografía

Filosofía visual

Las decisiones de diseño visual buscan transmitir calidez, apetito y energía sin perder la limpieza de una aplicación mobile-first, inspirándose en los tonos cálidos de la gastronomía.

Paleta de colores

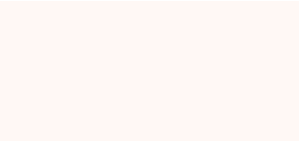
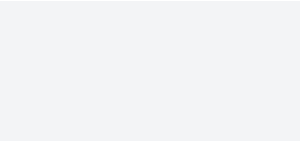
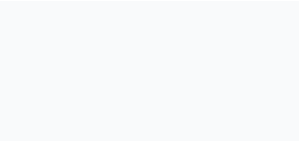
COLOR PRIMARIO

			
Primary #f45b3f Botones, activos, likes, nav	Primary dark #e24a2e Hover de botones	Primary light #ff8a66 Gradientes, fin de degradado	Orange-50 #fff3ef Fondos de badges y alertas

GRADIENTE PRINCIPAL

#f45b3f → #ff8a66 linear-gradient(135deg)	Cooking Journey & Login Tarjeta de progreso del home y cabecera del login
---	---

FONDOS Y SUPERFICIES

			
App background #fff8f5 Fondo global de la app	Card #ffffff Tarjetas y modales	Gray-100 #f3f4f6 Tags secundarios	Gray-50 #f9fafb Fondos internos de sección

ESCALA DE GRISES — TEXTOS

Clase Tailwind	Hex	Uso principal
gray-900	#111827	Títulos y textos principales
gray-800	#1f2937	Subtítulos destacados
gray-700	#374151	Labels de formulario
gray-600	#4b5563	Texto de descripción
gray-500	#6b7280	Metadatos y texto secundario
gray-400	#9ca3af	Hints, placeholders, iconos inactivos

COLORES SEMÁNTICOS

Error bg #fef2f2 Fondo mensajes de error	Error text #dc2626 Texto de error	Info bg #fff7ed Mensajes informativos

Tipografía

ESCALA TIPOGRÁFICA

Uso	Tamaño	Peso	Color
Título de aplicación	30px	700	gray-900
H2 destacado (hero)	24px	700	Blanco / gradiente
H2 de sección	20px	700	gray-900
Título de receta (card)	18px	700	gray-900
Cuerpo y descripciones	14px	400	gray-500
Metadatos y badges	12px	500	#f45b3f / gray-500
Subtítulos y etiquetas	11px	500	gray-400

Tokens de forma — border-radius

ESCALA DE REDONDEO

Componente	border-radius
Botones de acción y filtros de categoría	9999px (pill)
Secciones hero y panel Cooking Journey	32–36px
Tarjeta de receta (RecipeCard)	24–28px
Cards secundarias y contenedores de formulario	16px

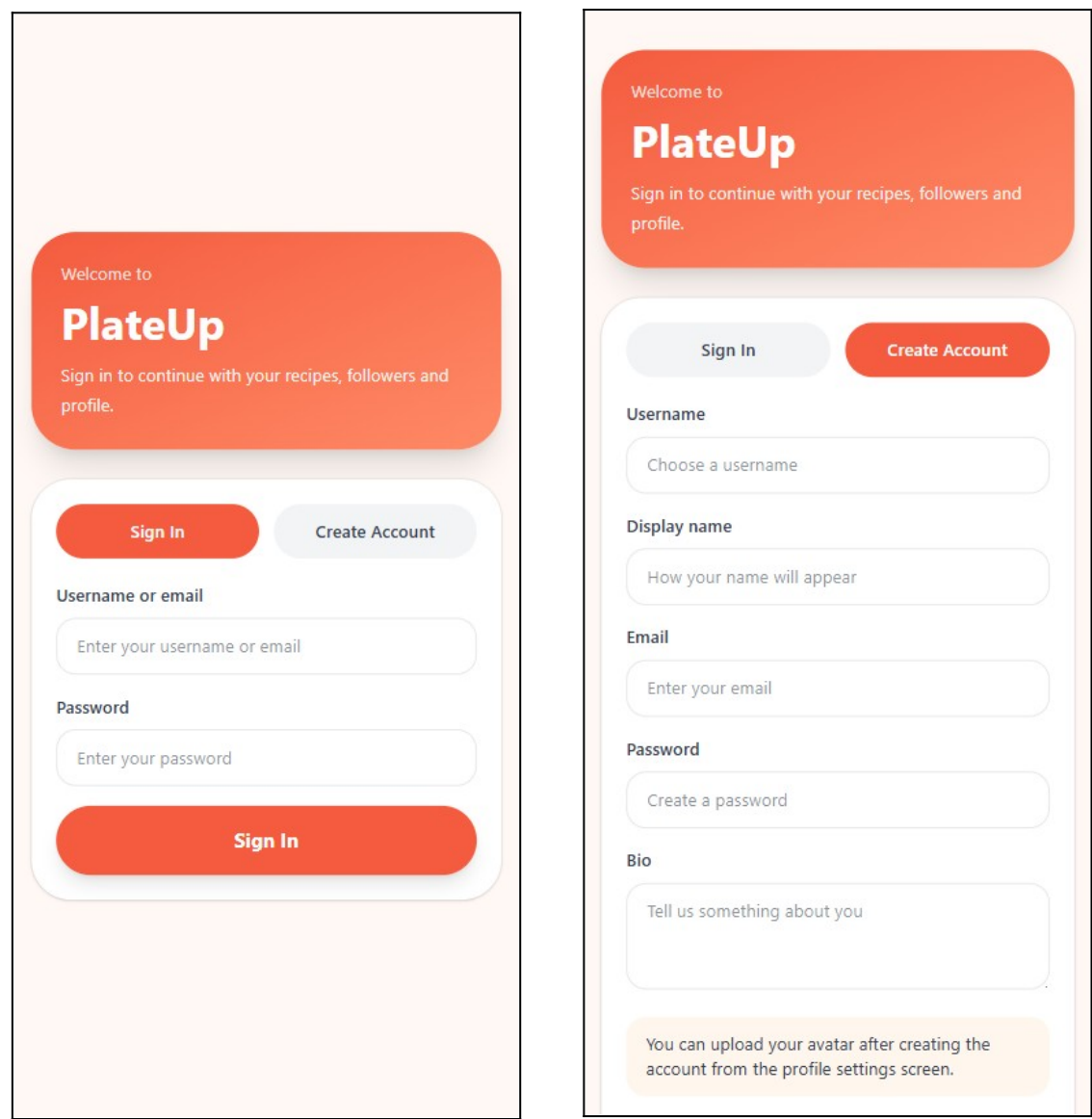
Elevación y separación

NIVELES DE SOMBRA

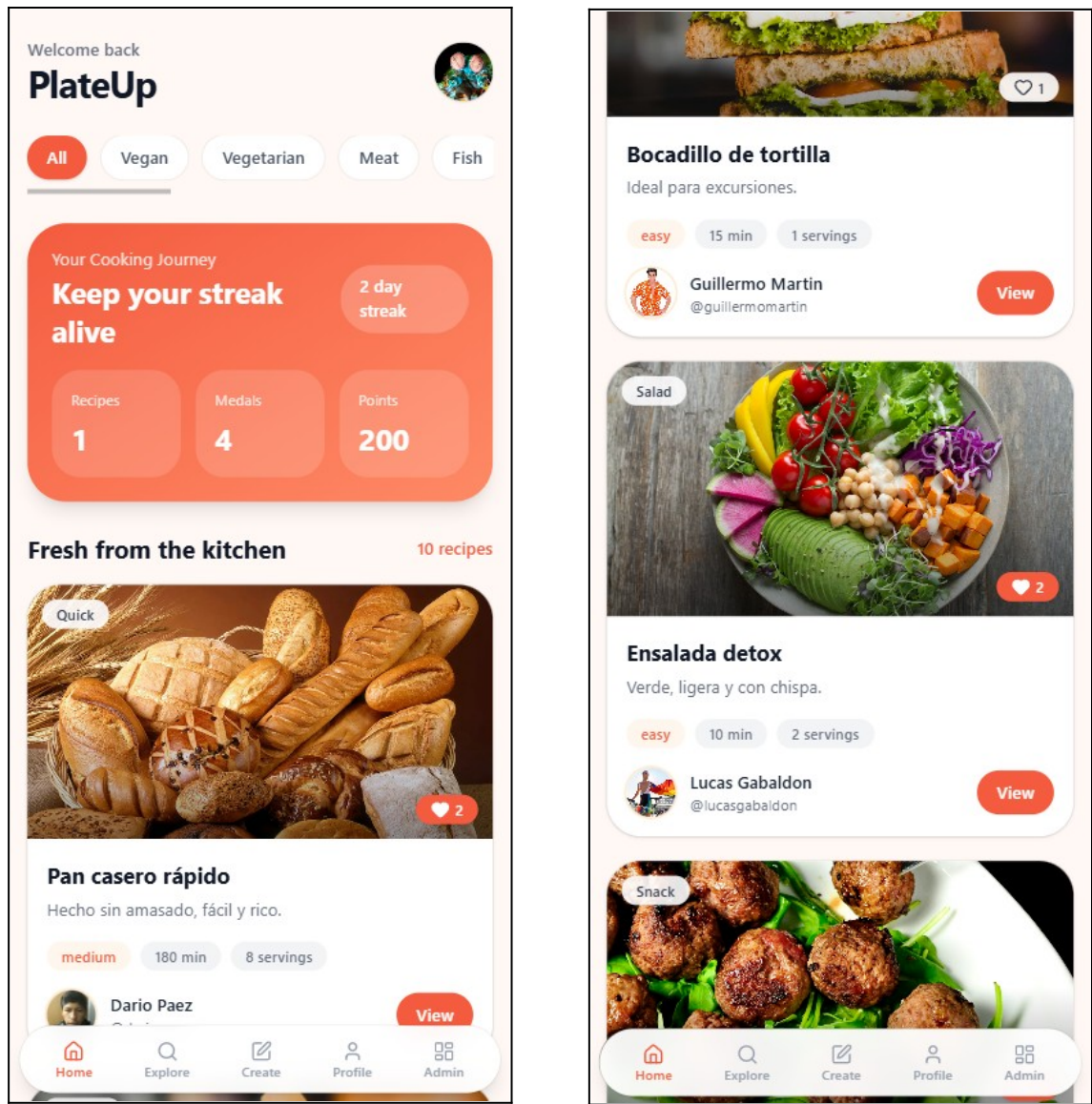
Nivel	Clase Tailwind	Uso
Borde fino	ring-1 ring-black/5	Tarjetas estándar, inputs
Sombra suave	shadow-sm	RecipeCard, listas
Sombra media	shadow-lg	Bloques hero, Cooking Journey
Sombra fuerte	shadow-xl	Login, BottomNav flotante

Anexo VIII. Capturas de pantalla de la interfaz

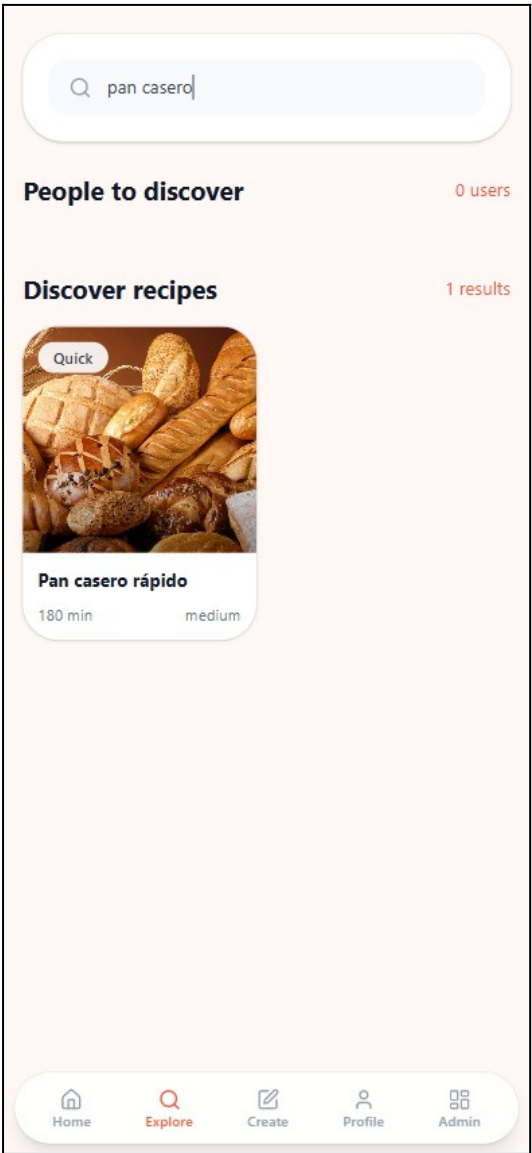
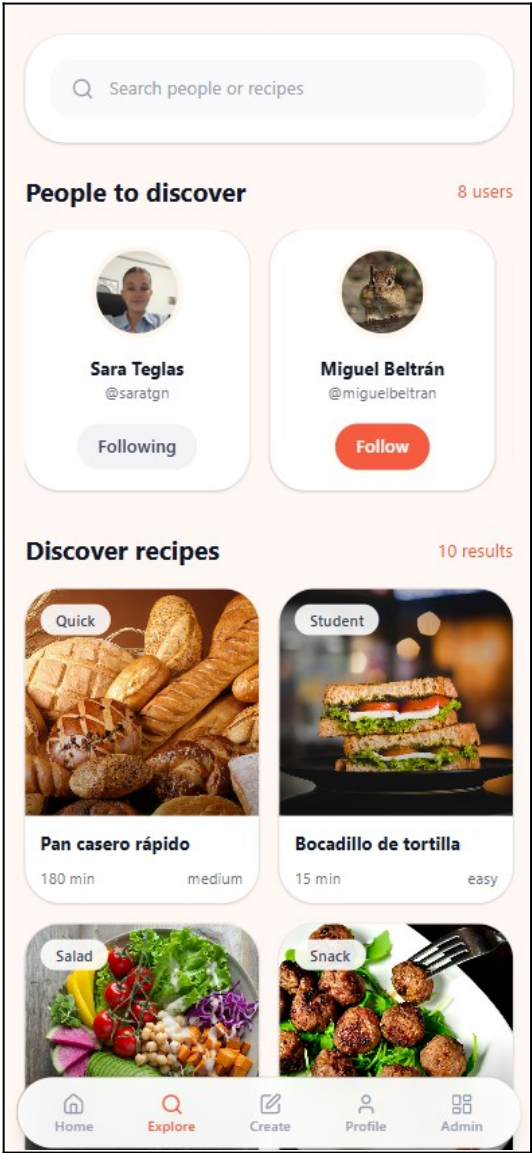
AuthView (Pantalla de autenticación)



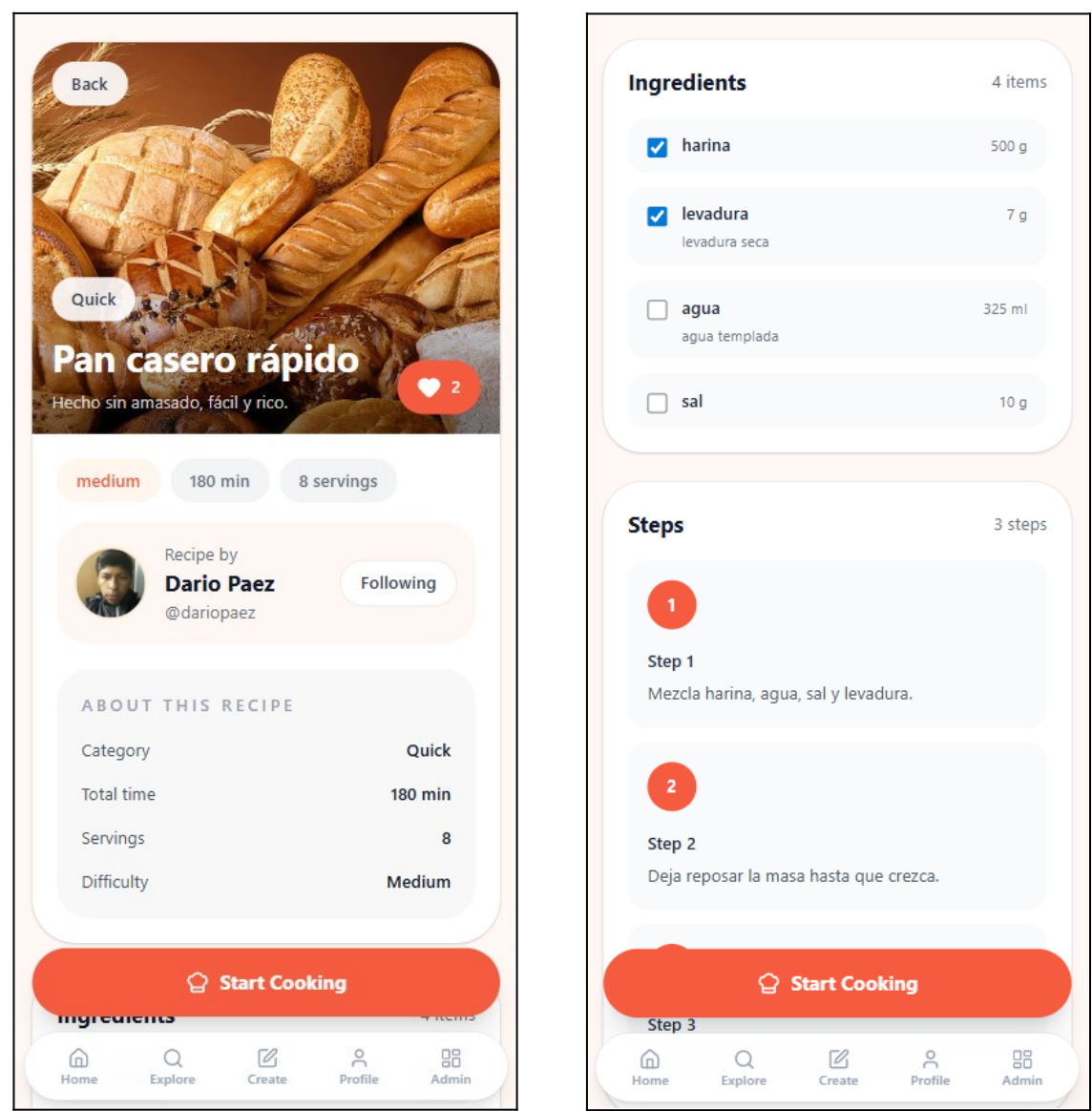
HomeView (Feed principal)



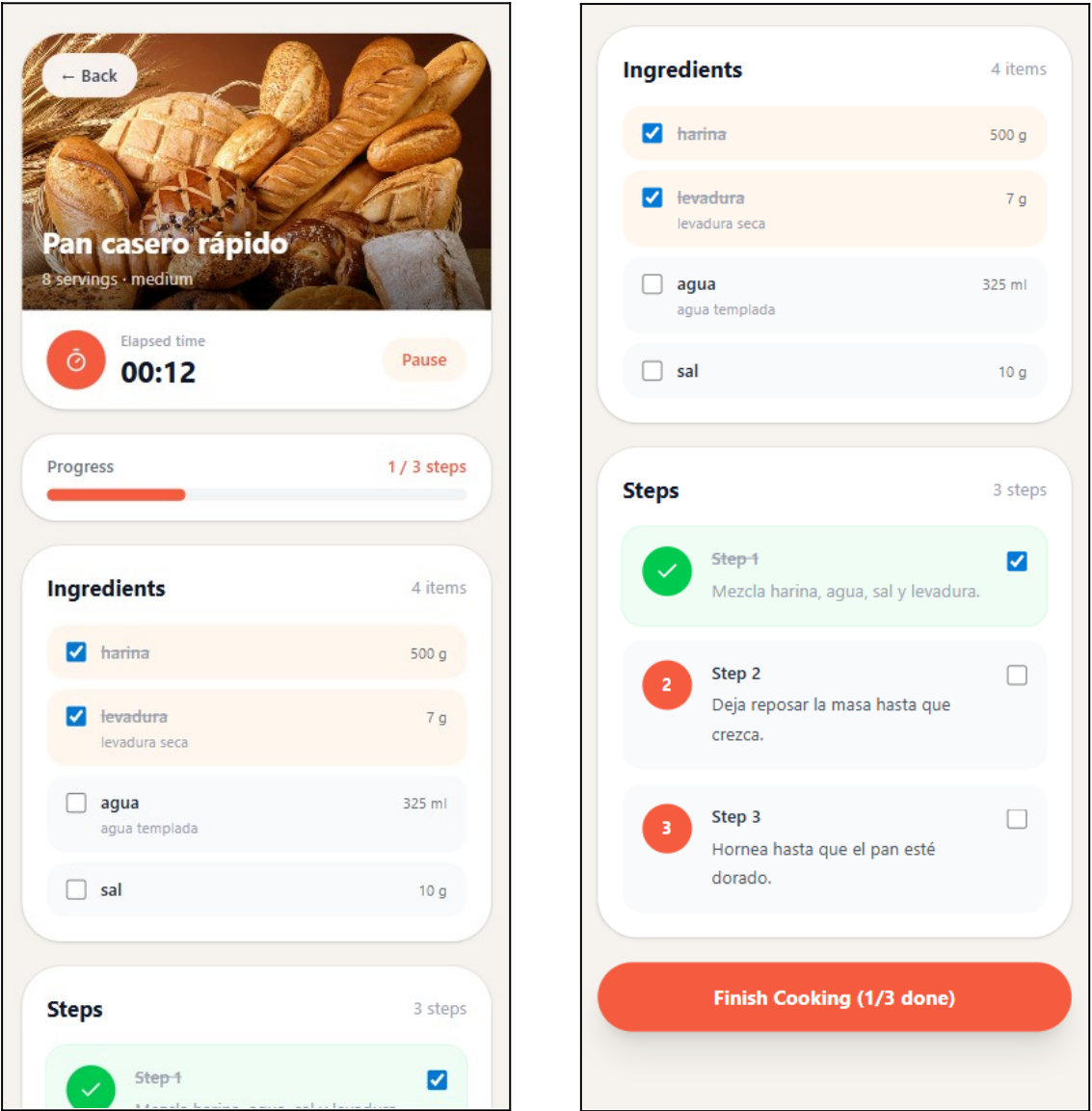
ExploreView (Exploración y búsqueda)



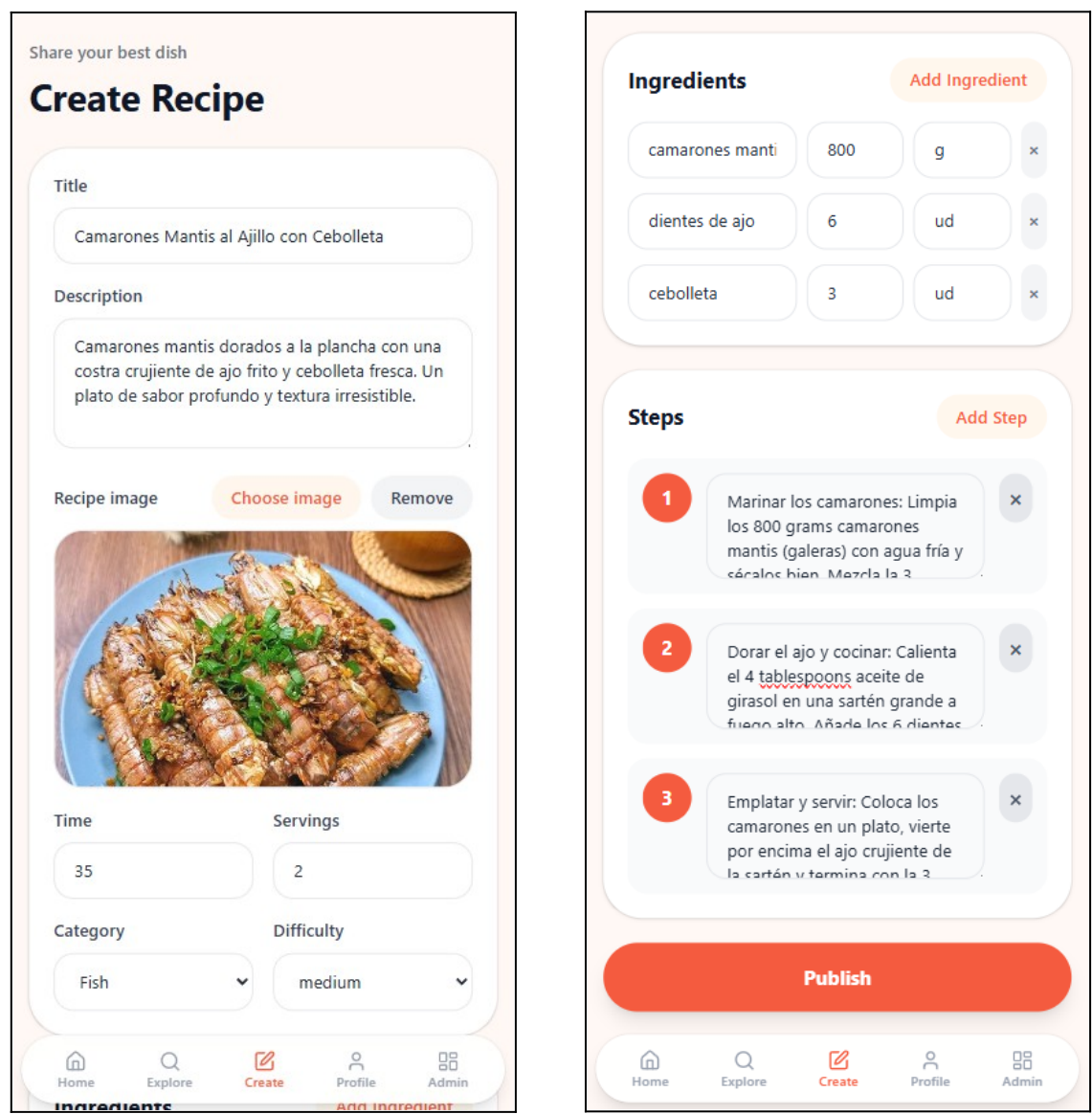
RecipeDetailView (Detalle de receta)



CookingView (Modo cocina guiado)



CreateView (Creación de receta)



ProfileView (Perfil propio)



Camarón Mantis

@camaron

Soy un camarón mantis.

Recipes

1

Followers

4

Following

9

My Recipe Book

Achievements

My Recipe Book



Pasta al pesto

25 min · easy

Delete recipe

Cooked Recipes

2




Home

Explore

Create

Profile

Admin



Camarón Mantis

@camaron

Soy un camarón mantis.

Recipes

1

Followers

4

Following

9

My Recipe Book

Achievements

Achievements



Tu primera receta

50 pts

Publica tu primera receta en PlateUp.

recipes



Primer aplauso

40 pts

Recibe tu primer like en una de tus recetas.

likes



Primera receta cocinada

50 pts

Termina de cocinar tu primera receta.

cooked



Tu primer seguidor

60 pts

Consigue tu primer seguidor.

followers

Home

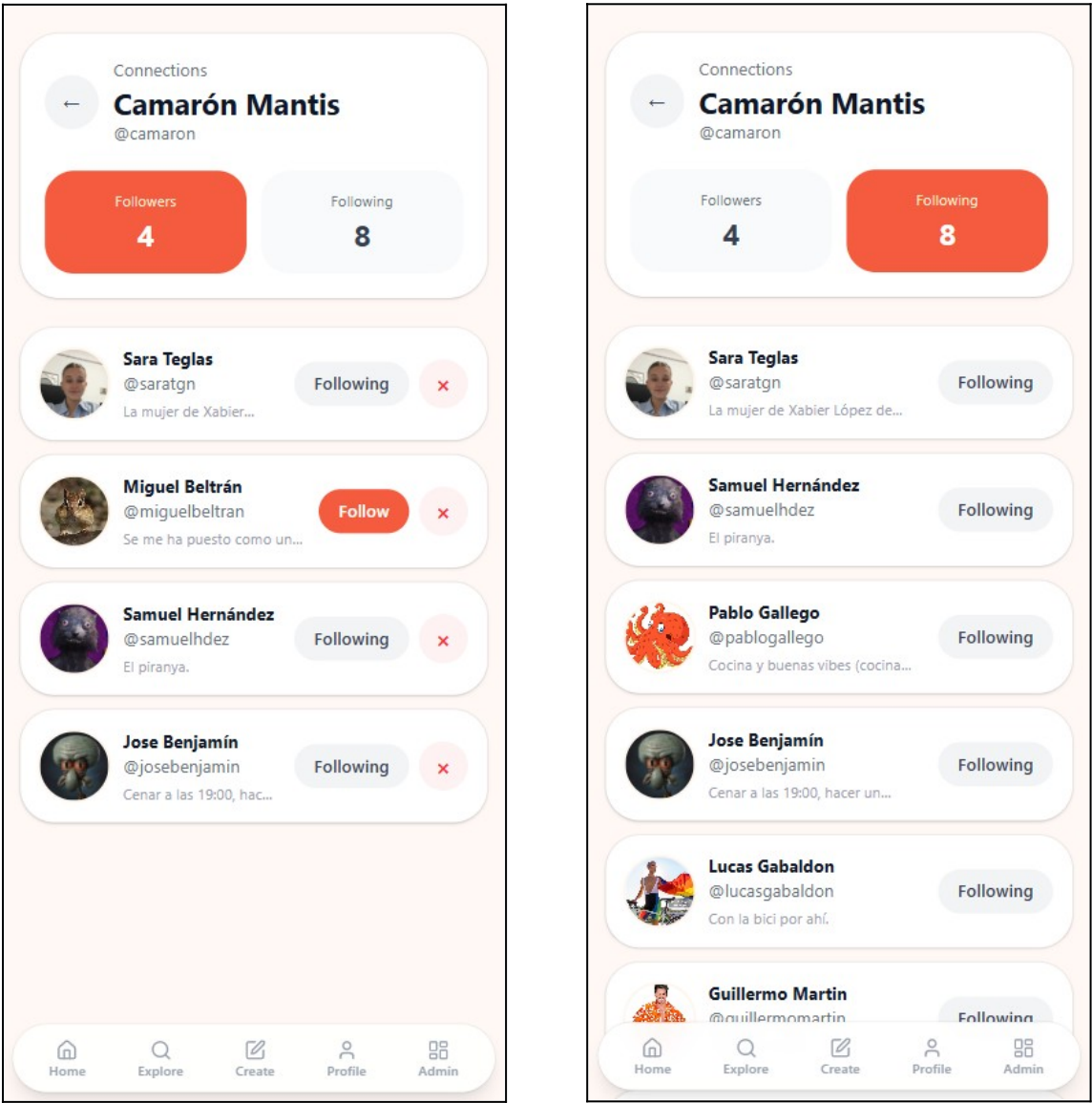
Explore

Create

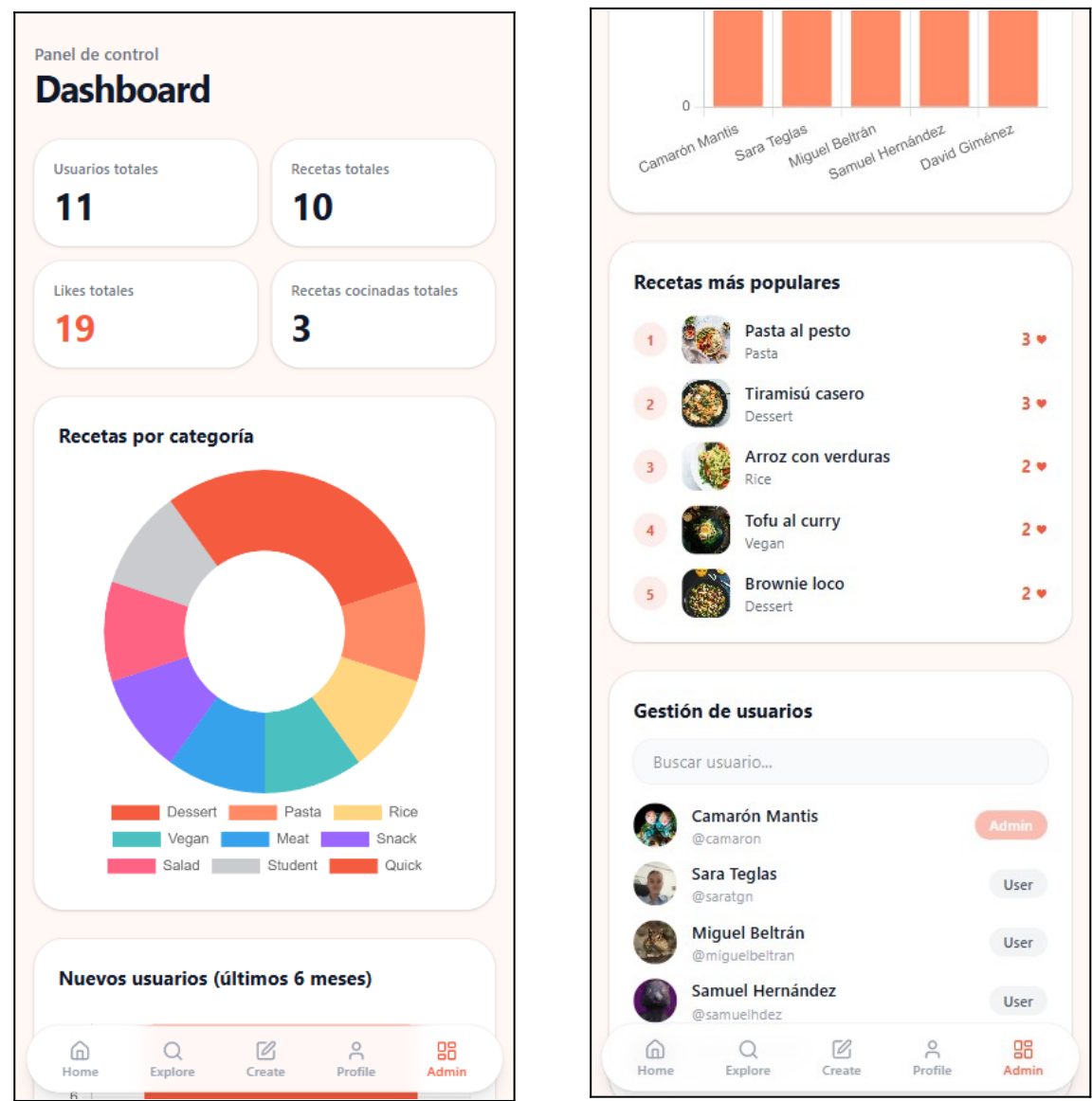
Profile

Admin

ConnectionsView (Panel de followers)



AdminDashboardView (Panel de administración)



Anexo IX. Repositorio de GitHub

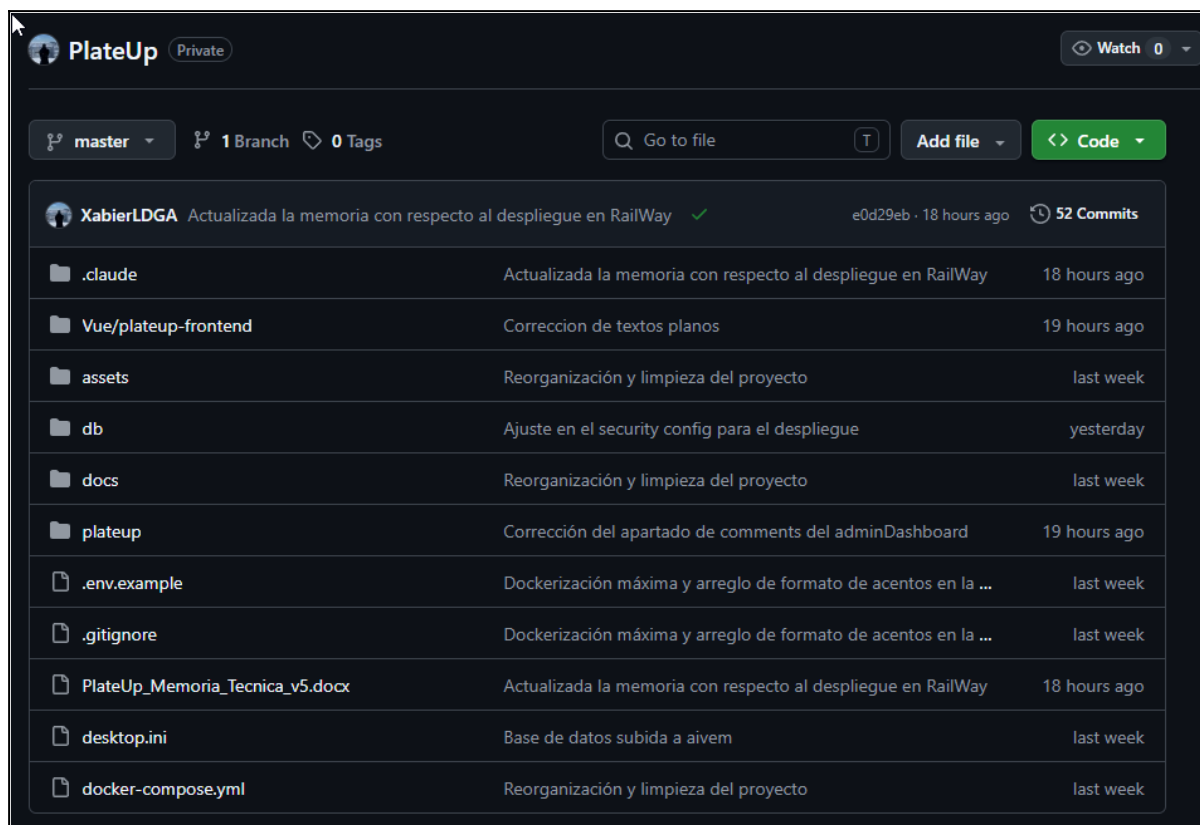
El repositorio de PlateUp está disponible públicamente en GitHub en la siguiente URL: <https://github.com/XabierLDGA/PlateUp.git>. El repositorio centraliza todo el código fuente del proyecto (backend, frontend y configuración de despliegue) y actúa además como punto de integración con Railway para el despliegue continuo automático.

Estructura del repositorio

El repositorio sigue una estructura monorepo, con la carpeta plateup/ (backend Spring Boot, Dockerfile, pom.xml), la carpeta plateup-frontend/ (Vue.js, package.json, vite.config.js, Capacitor) y, en la raíz, los archivos de configuración global (docker-compose.yml, railway.toml, nginx.conf).

Historial de commits y evolución del proyecto

El historial de commits refleja la metodología iterativa e incremental, con mensajes descriptivos que identifican la funcionalidad implementada en cada punto. La rama master contiene el historial completo desde la configuración inicial hasta la versión final desplegada en producción.



The screenshot shows the GitHub repository page for 'PlateUp' (Private). The interface includes a search bar, a 'Go to file' button, and a 'Code' button. The commit history is displayed as a table with columns for the commit message, the author, the commit hash, the time ago, and the number of commits.

Commit Message	Author	Commit Hash	Time Ago	Commits
Actualizada la memoria con respecto al despliegue en RailWay	XabierLDGA	e0d29eb	18 hours ago	52
.claude			18 hours ago	
Vue/plateup-frontend			19 hours ago	
assets			last week	
db			yesterday	
docs			last week	
plateup			19 hours ago	
.env.example			last week	
.gitignore			last week	
PlateUp_Memoria_Tecnica_v5.docx			18 hours ago	
desktop.ini			last week	
docker-compose.yml			last week	

Anexo X. Pruebas con Swagger UI

Swagger UI fue la herramienta principal utilizada para probar los endpoints de la API REST de PlateUp durante el desarrollo. Al estar integrada directamente en el proyecto mediante springdoc-openapi, permite probar cualquier endpoint desde el navegador sin necesidad de herramientas externas, utilizando la misma documentación generada automáticamente a partir de las anotaciones de los controladores.

Organización de la colección

Swagger UI organiza automáticamente todos los endpoints por controlador REST: Auth (registro y login), Users, Recipes, RecipeSteps, RecipeIngredients, Likes, Follows, Achievements, UserAchievements, Challenges, UserChallenges, Collections, Comments, CookedRecipes, Upload y Admin. Cada grupo muestra los métodos HTTP disponibles con su esquema de petición y respuesta, permitiendo ejecutar pruebas directamente desde el navegador.

Casos de prueba principales

Para cada endpoint se verificaron los siguientes casos: respuesta correcta con datos válidos (código 200 o 201 según corresponda), comportamiento ante datos inválidos o incompletos (código 400 Bad Request), comportamiento sin token JWT en endpoints protegidos (código 401 Unauthorized), comportamiento con token JWT inválido o expirado (código 401), e intento de acceder a recursos que no existen (código 404 Not Found). Los endpoints de administración se probaron adicionalmente con usuarios sin rol de administrador para verificar que el acceso se denegaba correctamente.

Servers	
http://localhost:8080 - Generated server url	
utensil-controller	▼
user-controller	▼
user-challenge-controller	▼
user-achievement-controller	▼
recipe-step-controller	▼
recipe-controller	▼
ingredient-controller	▼
follow-controller	▼
comment-controller	▼
collection-controller	▼
collection-recipe-controller	▼
challenge-controller	▼
achievement-controller	▼
upload-controller	▼
recipe-ingredient-controller	▼
like-controller	▼
cooked-recipe-controller	▼
admin-controller	▼

Resultados de las pruebas

Todos los endpoints de la API REST superaron las pruebas con el resultado esperado en ambos entornos: local (localhost:8080) y producción (Railway). Los 18 controladores respondieron correctamente con los códigos HTTP apropiados, los cuerpos de respuesta JSON bien formados y el comportamiento correcto de la autenticación JWT.

The screenshot displays a REST client interface with the following details:

- Method:** POST
- URL:** /api/users/register
- Parameters:** No parameters
- Request body:** application/json
- Request body content:**

```
{  "username": "testuser",  "email": "testuser@plateup.com",  "password": "Password123!",  "displayName": "Usuario de Prueba",  "bio": "Me encanta cocinar recetas mediterráneas.",  "avatarUrl": "",  "visibilityDefault": "public"}
```
- Execute:** Button to execute the request.
- Responses:**
 - Curl:**

```
curl -X 'POST' \  'http://localhost:8080/api/users/register' \  -H 'accept: */*' \  -H 'Content-Type: application/json' \  -d '{  "username": "testuser",  "email": "testuser@plateup.com",  "password": "Password123!",  "displayName": "Usuario de Prueba",  "bio": "Me encanta cocinar recetas mediterráneas.",  "avatarUrl": "",  "visibilityDefault": "public"  }'
```
 - Request URL:** http://localhost:8080/api/users/register
 - Server response:**

Code	Details
201	Response body <pre>{ "token": "eyJhbGciOiJIUzI1NiIsInR5cGU6IjY9LmVzZXJ7ZC6MTIsImVtYWlsIjoiaWdGVzdhVzZlZC6GxhdGV1cC5jb201LCJyb2x1IjoiaWVtYUlsImhhdCI6MTc3OTM1ODc2OiwiaWF0IjoxNzgzOTUwNzN4FQ.DFTvmgU47JDDQqWNCJg_F2vNsZ1A1P0sCcphwK-8stFP0_YbsNdLGAUV03uvCuaVzMCnvojTrPKUjKwqQ1Yw", "user": { "id": 12, "username": "testuser", "email": "testuser@plateup.com", "displayName": "Usuario de Prueba", "bio": "Me encanta cocinar recetas mediterráneas.", "avatarUrl": "", "visibilityDefault": "public", "createdAt": "2026-05-21T12:18:58.7059023", "updatedAt": "2026-05-21T12:18:58.7059023", "streakCount": 0, "lastActiveDate": null, "role": "USER" } }</pre>